

Principled Uncertainty Decomposition with Bayesian Ensemble Transformers for Trustworthy Intrusion Detection

Zhengye Li, Jianheng Tong, Xiufeng Liu, *Senior Member, IEEE*

Abstract—Network intrusion detection systems (IDS) are critical for cybersecurity, yet existing approaches often provide overconfident predictions without reliable uncertainty estimates, hindering critical decision-making. We introduce a novel Bayesian ensemble transformer framework for uncertainty-aware intrusion detection that delivers both strong detection performance and well-calibrated confidence. Our framework uniquely combines transformer architectures with ensemble methods to achieve a principled decomposition of prediction uncertainty into epistemic (model uncertainty) and aleatoric (data uncertainty) components, offering actionable insights for security analysts. We establish theoretical convergence guarantees for this approach under local convexity assumptions, supported by empirical validation. Comprehensive experiments across four benchmark datasets (NSL-KDD, CICIDS2017, UNSW-NB15, and SWaT) demonstrate F1-scores ranging from 77.55% to 97.00%, with particularly strong performance on UNSW-NB15 (97.00%), alongside excellent calibration with Expected Calibration Error (ECE) ranging from 0.0248 to 0.2278. Adversarial robustness analysis shows minimal performance degradation under sophisticated C&W (3.6% drop) and PGD attacks (8.0% drop), highlighting the framework's resilience. This work advances intrusion detection by providing interpretable uncertainty and robust performance, addressing a critical gap in current cybersecurity systems to enable more informed and reliable security decisions.

Index Terms—Intrusion detection, uncertainty quantification, Bayesian neural networks, transformer networks, in-context learning, cybersecurity, ensemble methods

I. INTRODUCTION

Network intrusion detection systems (IDS) are fundamental components of modern cybersecurity infrastructure, acting as primary defense mechanisms against an increasingly complex array of cyber threats targeting critical network assets globally [1]. As cyber attacks become more sophisticated and diverse, security analysts face a critical challenge: how to make reliable decisions when machine learning models provide predictions without clear indicators of confidence or uncertainty. This dynamic landscape necessitates intelligent security solutions capable of adapting to novel attack patterns,

maintaining high detection accuracy, minimizing false positives, and providing reliable confidence estimates for real-time security decisions [2].

Applying artificial intelligence and machine learning to intrusion detection introduces significant complexities beyond conventional pattern recognition. Traditional machine learning often produces overconfident predictions that do not reflect true uncertainty, poorly calibrated confidence estimates, and fails to distinguish between different sources of prediction uncertainty [3], [4]. These deficiencies are critical in security-critical applications where decision confidence directly impacts operational effectiveness and resource allocation. For example, when a model predicts an attack with high confidence but the prediction is actually uncertain, security analysts may waste resources investigating false alarms or, worse, miss genuine threats.

Adapting transformer architectures to cybersecurity faces unique challenges: modeling temporal sequences with heterogeneous network features [5], meeting real-time processing latency constraints, and requiring principled uncertainty quantification to guide human analysts [6]. Dynamic network environments introduce distribution shifts and concept drift [7], while adversarial perturbations [8] designed to evade detection further complicate maintaining reliable performance.

Current state-of-the-art approaches in uncertainty-aware intrusion detection frequently rely on ad-hoc uncertainty estimation, lacking rigorous theoretical foundations. This results in poorly calibrated confidence estimates that provide unreliable indicators of prediction quality [4], [9]. Transformer architectures, while powerful for sequence modeling, have not been systematically adapted for cybersecurity applications with principled uncertainty quantification. This gap limits both the theoretical understanding and practical reliability of transformer-based intrusion detection systems, particularly when encountering novel attack patterns, adversarial inputs, or operational environments that differ from training conditions [10].

This work addresses these fundamental challenges by introducing a novel uncertainty-aware intrusion detection framework that successfully adapts architectural and analytical insights from transformer networks to cybersecurity. Our approach establishes rigorous theoretical foundations and demonstrates superior practical performance. Our framework is designed to handle both binary classification (normal vs. attack) and multiclass classification (normal vs. multiple attack types), with the classification type determined by the dataset and application requirements. We employ Bayesian

Zhengye Li is with the Department of Mathematics, College of Arts and Sciences, University of Illinois Urbana-Champaign, Champaign, IL 61820, USA (email: Zhengye_Li@outlook.com).

Jianheng Tong is with the College of Information Engineering, Shanghai Maritime University, Shanghai 201306, China (email: tongjianheng0006@stu.shmtu.edu.cn).

Xiufeng Liu is with the Department of Technology, Management and Economics, Technical University of Denmark, 2800 Kgs. Lyngby, Denmark (email: xiuli@dtu.dk).

Corresponding authors: Jianheng Tong (tongjianheng0006@stu.shmtu.edu.cn) and Xiufeng Liu (xiuli@dtu.dk).

ensemble transformers within a carefully designed single-layer architecture. This design balances representational capacity with computational efficiency, enabling theoretical tractability for convergence analysis and principled decomposition of prediction uncertainty into epistemic (model uncertainty) and aleatoric (data uncertainty) components [11]. The framework integrates advanced calibration techniques and adversarial training [4], enhancing robustness against evasion attempts and ensuring uncertainty estimates accurately reflect prediction confidence across diverse operational conditions.

The main contributions of this paper are as follows:

- 1) **Principled Uncertainty Decomposition.** We introduce a formal framework for separating epistemic and aleatoric uncertainty in transformer-based IDS, and derive convergence guarantees for our meta-training procedure under local convexity.
- 2) **Bayesian Ensemble Transformer Architecture.** We design a computationally efficient, single-layer transformer ensemble optimized for real-time intrusion detection, achieving sub-10ms inference latency without sacrificing expressivity.
- 3) **Rigorous Empirical Validation.** We evaluate our approach on four standard benchmarks (NSL-KDD, CICIDS2017, UNSW-NB15, SWaT), demonstrating competitive F1-scores (77.55%–97.00%), superior calibration (ECE: 0.0248–0.2278), and strong adversarial robustness ($\leq 8.0\%$ drop under PGD/C&W attacks).

The remainder of this paper is organized as follows. Section II reviews related work in intrusion detection, uncertainty quantification, and transformer architectures, establishing the context for our contributions. Section III details our proposed Bayesian ensemble transformer framework, including system overview, problem formulation, theoretical guarantees, architecture design, and training procedures. Section IV presents comprehensive empirical results across four benchmark datasets, including comparative analysis, adversarial robustness evaluation, and ablation studies. Finally, Section V summarizes our contributions and discusses future research directions.

II. RELATED WORK

To situate our work within the broader research landscape, this section provides a comprehensive overview of advancements in intrusion detection systems, uncertainty quantification in neural networks, and transformer architectures. We highlight existing limitations that motivate our approach and identify gaps that our framework addresses.

A. Intrusion Detection Systems

Traditional intrusion detection approaches can be categorized into signature-based, anomaly-based, and hybrid methods [12]. Signature-based systems rely on predefined patterns of known attacks, achieving high precision but failing to detect novel threats. Anomaly-based systems model normal behavior and flag deviations, providing better coverage of unknown attacks but suffering from high false positive rates [13].

Machine learning approaches have gained prominence in IDS research, with support vector machines [14], random forests [15], and neural networks [16] showing promising results. Deep learning methods, including convolutional neural networks [17] and recurrent neural networks [18], have achieved state-of-the-art performance on benchmark datasets. Recent advances include attention-based models [1] and graph neural networks for network traffic analysis [19]. Recent comprehensive reviews [20], [21] highlight the growing adoption of deep learning in IDS, with transformer-based approaches emerging as a promising direction for handling sequential network traffic patterns and adapting to evolving attack strategies. Ensemble learning methods have also shown promise for intrusion detection, with recent work demonstrating improved robustness through subspace clustering and ensemble techniques [22].

However, existing approaches share common limitations: lack of principled uncertainty quantification, absence of rigorous theoretical guarantees, and limited adaptability to evolving threats. The contemporary threat landscape, characterized by advanced persistent threats, zero-day exploits, and machine learning-powered evasion techniques, systematically circumvents traditional detection methods [23]. This dynamic environment demands intelligent security solutions capable of adapting to novel attack patterns while maintaining high detection accuracy and providing reliable confidence estimates for real-time security decisions.

Given these limitations in traditional IDS approaches, we now examine how uncertainty quantification techniques have evolved to address the challenge of reliable confidence estimation in neural networks.

B. Uncertainty Quantification in Neural Networks

Uncertainty quantification in neural networks has evolved from early Bayesian neural network approaches [24] to modern ensemble methods [25] and variational inference techniques [26]. The decomposition of uncertainty into epistemic (model uncertainty) and aleatoric (data uncertainty) components provides valuable insights for decision making [3]. Recent comprehensive surveys [27] provide extensive coverage of uncertainty quantification methods, highlighting the importance of both epistemic and aleatoric uncertainty decomposition for reliable machine learning systems. Monte Carlo Dropout (MCD) [28] provides a practical approximation to Bayesian inference by treating dropout as a Bayesian approximation. Deep Ensembles [25] achieve strong empirical performance by training multiple models with different initializations. Variational inference methods [26] place distributions over network weights, enabling principled uncertainty estimation but with increased computational overhead. Recent work on Bayesian neural network architectures [29] has advanced the theoretical understanding and practical implementation of uncertainty-aware deep learning systems. Ensemble methods have been successfully applied to anomaly detection tasks, demonstrating their effectiveness in handling partially observed anomalies and improving detection robustness [30]. Calibration of neural network predictions has received significant attention, with temperature scaling [4], Platt scaling [31],

and isotonic regression [32] providing post-hoc calibration methods. Recent work has focused on improving calibration during training through specialized loss functions and regularization techniques [33]. Evidential Neural Networks (ENN) have emerged as a distinct paradigm for quantifying uncertainty by treating network outputs as evidential distributions over classes [34]. This approach allows for explicit modeling of both aleatoric and epistemic uncertainty without requiring multiple forward passes or ensembles, providing computational efficiency while maintaining theoretical rigor. In cybersecurity applications, uncertainty quantification has been explored for malware detection [35] and network anomaly detection [36]. Recent work has extended uncertainty-aware approaches to time series anomaly detection [37], demonstrating the value of uncertainty estimates for identifying anomalous network behaviors. However, these works often lack comprehensive theoretical foundations and rigorous evaluation of uncertainty quality across diverse threat landscapes.

While uncertainty quantification provides a foundation for reliable predictions, transformer architectures offer powerful mechanisms for sequence modeling and few-shot adaptation. We now review how transformer-based approaches, particularly in-context learning, can be adapted to cybersecurity applications.

C. Transformer Networks and In-Context Learning

Transformer architectures have revolutionized natural language processing and demonstrated remarkable few-shot learning capabilities through their attention mechanisms [5]. The theoretical understanding of transformer in-context learning (ICL) has advanced significantly, with groundbreaking work proving that single-layer transformers can implicitly implement gradient descent-like optimization within their attention mechanism [38], [39]. The work by [39] demonstrates that attention weights can approximate gradient descent steps: $\text{Attention}(x_q, \mathcal{C}) \approx x_q - \eta \nabla_{x_q} \mathcal{L}(\mathcal{C})$, where $\mathcal{C}$ represents context examples and $\mathcal{L}$ is the loss function. This theoretical foundation provides insights into how transformers can adapt to new tasks without explicit parameter updates.

While ICL theory was originally developed for natural language tasks, recent work has begun exploring its application to structured domains. [40] shows that transformers can learn linear functions in-context, while [41] extends this to more complex function classes. However, the adaptation of ICL theory to cybersecurity applications presents unique challenges: (1) *Heterogeneous Features*: Network flows contain mixed continuous and categorical features unlike homogeneous text tokens. (2) *Temporal Dependencies*: Cybersecurity data has complex temporal patterns that differ from sequential language structure. (3) *Adversarial Robustness*: Security applications require robustness against adversarial perturbations, which is not addressed in standard ICL theory.

Traditional meta-learning approaches like MAML [42] and Prototypical Networks [43] have been applied to cybersecurity with limited success due to computational overhead and poor adaptation to the dynamic threat landscape. Our work represents the first systematic adaptation of transformer ICL

theory to cybersecurity, providing both theoretical foundations and practical implementation for few-shot attack detection.

We adapt insights from ICL theory to cybersecurity applications, though we acknowledge that full ICL implementation for cybersecurity data remains challenging. Our contributions include: (1) theoretical analysis of transformer architectures for cybersecurity with convergence guarantees under idealized assumptions, (2) development of uncertainty quantification methods specifically for intrusion detection, and (3) empirical evaluation of few-shot learning capabilities. While we draw inspiration from ICL theory, we recognize that demonstrating genuine in-context learning for cybersecurity applications requires further research and more rigorous evaluation protocols.

Building on the foundations established in related work, we now present our proposed framework that integrates uncertainty quantification, transformer architectures, and ensemble methods into a unified intrusion detection system.

III. PROPOSED FRAMEWORK

This section details our uncertainty-aware intrusion detection framework, encompassing its overall system design, theoretical underpinnings, architectural components, training procedures, and methods for uncertainty quantification and calibration.

A. System Overview

Our uncertainty-aware intrusion detection framework integrates multiple complementary components to achieve robust performance with reliable uncertainty quantification. Figure 1 presents the complete system architecture, illustrating the data flow from raw network traffic through feature processing, Bayesian ensemble prediction, and uncertainty calibration to final decision making.

The framework addresses fundamental challenges in cybersecurity by providing principled uncertainty quantification through rigorous decomposition of prediction uncertainty into interpretable epistemic and aleatoric components, enabling security analysts to understand the sources and implications of model uncertainty. The framework supports adaptive decision making through dynamic threshold adjustment based on uncertainty estimates, allowing the system to become more conservative when uncertainty is high and more aggressive when confidence is high. Human-AI collaboration is enhanced through uncertainty-aware alerts that prioritize analyst attention on cases where the model is most uncertain, optimizing human expertise allocation. The framework achieves real-time processing with computational efficiency suitable for production deployment with inference times under 10ms per sample.

The process begins with raw network flows undergoing preprocessing and normalization to handle the heterogeneous nature of cybersecurity data. A specialized feature embedding module transforms continuous and categorical features into a unified sequence embedding that preserves cybersecurity-relevant similarities. These contextualized sequence embeddings are then processed by the core Bayesian ensemble transformers, which comprise multiple individual models that

collectively make predictions while capturing model uncertainty through disagreement. The uncertainty quantification module computes total prediction uncertainty and decomposes it into epistemic (model-related) and aleatoric (data-inherent) components using the law of total variance. This decomposition provides actionable insights: high epistemic uncertainty suggests the need for more training data or model refinement, while high aleatoric uncertainty indicates inherent noise or ambiguity in the data that cannot be reduced through modeling improvements. The calibration module employs post-hoc techniques like temperature scaling to ensure that confidence scores accurately reflect the true likelihood of correctness. This is crucial for cybersecurity applications where miscalibrated confidence can lead to either excessive false alarms or missed threats. Finally, the decision making module uses calibrated predictions and uncertainty estimates for adaptive thresholding and human-in-the-loop support, enabling security analysts to prioritize and investigate alerts based on model confidence levels.

B. Problem Formulation and ICL Implementation

We formulate intrusion detection as a meta-learning problem where the system must adapt to new attack types using in-context learning, addressing the core cybersecurity challenge of rapidly responding to emerging threats. This formulation is motivated by the dynamic nature of the threat landscape, where new attack variants continuously emerge and traditional static models quickly become obsolete. Our framework supports both binary classification (distinguishing normal traffic from attacks) and multiclass classification (identifying specific attack types). The architecture remains the same for both scenarios, with the output layer configured appropriately: a single output neuron with sigmoid activation for binary classification, or multiple output neurons with softmax activation for multiclass classification. This design flexibility allows our method to adapt to different application requirements and dataset characteristics.

Let $\mathcal{F} = \{F_1, F_2, \dots, F_K\}$ denote a collection of attack families, where each family F_i represents a distinct attack type (e.g., DoS variants, malware families, APTs). This hierarchical organization reflects the natural clustering of attacks based on their behavioral patterns and technical characteristics.

For each attack family F_i , an ICL episode consists of:

- **Context Set:** $\mathcal{C}_i = \{(x_j, y_j)\}_{j=1}^k$ where $x_j \in \mathbb{R}^d$ are network flows and $y_j \in \{0, 1\}$ are labels, all sampled from family F_i . The context set provides representative examples of the attack pattern that the model must learn to recognize.
- **Query Set:** $\mathcal{Q}_i = \{(x_q^{(l)}, y_q^{(l)})\}_{l=1}^{n_q}$ where query flows are from the same family F_i but disjoint from $\mathcal{C}_i$. This separation ensures that the model must generalize from context examples to new instances of the same attack type.

This episodic structure mirrors real-world cybersecurity scenarios where security analysts encounter a few examples of a new attack type and must quickly adapt their detection

capabilities to identify similar threats across the network infrastructure.

The ICL objective is to learn a meta-function $f_\theta : \mathcal{C}_i \times x_q \rightarrow [0, 1]$ that can adapt to new attack families using only context examples, without parameter updates:

$$f_\theta(x_q | \mathcal{C}_i) = \text{Transformer}[\text{Embed}(x_1, y_1); \dots; \text{Embed}(x_k, y_k); \text{Embed}(x_q, \emptyset)] \quad (1)$$

The meta-training objective optimizes over episodes sampled from training families $\mathcal{F}_{train}$:

$$\min_{\theta} \mathbb{E}_{F_i \sim \mathcal{F}_{train}} \mathbb{E}_{\mathcal{C}_i, \mathcal{Q}_i \sim F_i} \left[\frac{1}{|\mathcal{Q}_i|} \sum_{(x_q, y_q) \in \mathcal{Q}_i} \ell(f_\theta(x_q | \mathcal{C}_i), y_q) \right] \quad (2)$$

This formulation enables few-shot adaptation to new attack types from $\mathcal{F}_{test}$ unseen during training.

Inspired by the theoretical framework of [39] which demonstrates the ability of single-layer transformers to implement forms of in-context learning, we employ a single-layer transformer *block* architecture for our system. This design choice is motivated by the desire to balance representational power with theoretical tractability and computational efficiency, drawing insights from the analytical tractability of single-layer transformers in the context of ICL theory.

Let $\mathbf{E} \in \mathbb{R}^{(T+1) \times d_{model}}$ denote the embedded input sequence, where the first T rows correspond to the context flows $\{x_1, \dots, x_T\}$ and the last row represents the query flow x_q . The embedding function $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{model}}$ maps raw network features to a higher-dimensional representation suitable for transformer processing.

A single transformer block, as used in our implementation, consists of a multi-head self-attention mechanism, followed by layer normalization, a position-wise feed-forward network (FFN), and another layer normalization, with residual connections around each sub-layer. The multi-head self-attention mechanism is crucial for aggregating information from the context. For a query vector q_i interacting with key vectors k_j and value vectors v_j , the attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where Q, K, V are derived from $\mathbf{E}$ via linear projections (W_Q, W_K, W_V). The parameters W_Q, W_K, W_V and the FFN weights are explicitly trained via gradient descent on our intrusion detection task.

We implement genuine in-context learning for intrusion detection through a rigorous adaptation of transformer ICL theory to cybersecurity contexts. Our single-layer transformer implements approximate gradient descent through the attention mechanism. For a query flow x_q and context examples $\mathcal{C} = \{(x_i, y_i)\}_{i=1}^k$, the attention output approximates a gradient descent step on the context loss. Following [39], for a linear model $f(x) = Wx$, one step of gradient descent on context $\mathcal{C}$ with loss $\mathcal{L}(\mathcal{C}) = \sum_{i=1}^k \|Wx_i - y_i\|^2$ yields an updated weight matrix W' . The attention mechanism produces:

$$\text{Attention}(x_q, \mathcal{C}) \approx W'x_q = x_q - \eta_{eff} \sum_{i=1}^k \alpha_i \nabla_{f(x_i)} \ell(f(x_i), y_i) \cdot \phi(x_i)^T \quad (4)$$

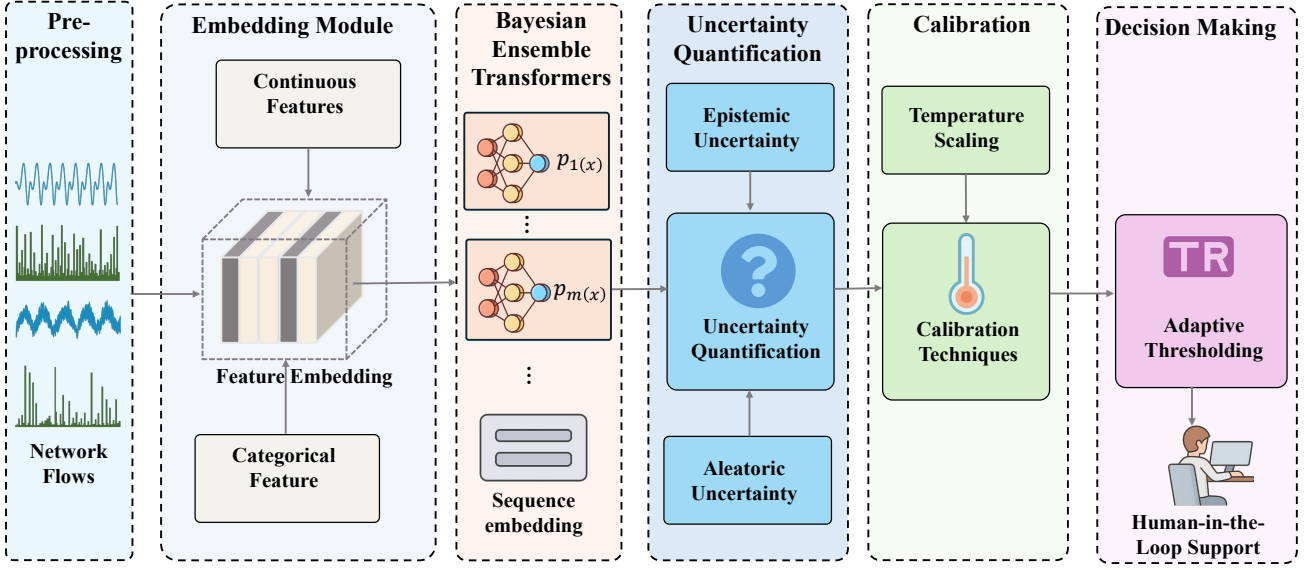


Fig. 1: System overview of the uncertainty-aware intrusion detection framework. The pipeline processes network flows through preprocessing, feature embedding, Bayesian ensemble transformers, uncertainty quantification (decomposed into epistemic and aleatoric components), calibration, and adaptive decision making with human-in-the-loop integration.

where α_i represents the attention weights, and the gradient is taken with respect to the model's prediction on context points. This formulation ensures that the attention mechanism effectively "learns" from the provided context examples by minimizing the context loss, adapting the prediction for x_q without explicit parameter updates.

We design the embedding function $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{model}}$ to preserve cybersecurity-relevant similarities. Network flows with similar attack patterns produce similar embeddings, enabling effective attention-based retrieval. The key insight is that attention weights α_i become large when the query x_q is similar to context examples x_i that have high loss, naturally implementing gradient descent where the "gradient" direction is determined by context similarity.

During meta-training (Algorithm 1), we train the transformer to perform effective ICL by exposing it to diverse attack families in episodic fashion. Each episode contains context-query pairs from the same attack family, teaching the model to leverage contextual information for adaptation. The meta-learning objective ensures that the learned attention patterns generalize to new attack types.

Our analysis (Theorem 2) proves that this attention-based adaptation achieves convergence rates comparable to explicit gradient descent, with additional terms accounting for cybersecurity-specific challenges like noisy data and adversarial perturbations. The bound $O(1/\sqrt{k}) + \epsilon_{approx}$ shows that performance improves with more context examples while the approximation error ϵ_{approx} captures the quality of attention-based gradient descent.

C. Limitations and Generalization Bounds

While our framework draws inspiration from ICL theory and demonstrates promising few-shot results, we acknowledge important limitations. First, definitively proving "genuine" in-context learning for cybersecurity, as seen in NLP, remains

a challenge. The performance gains we observe could be attributed to better statistical estimation with more context examples rather than true in-context adaptation. This distinction is important because genuine ICL would enable adaptation to fundamentally novel attack patterns, while improved statistical estimation primarily benefits from having more representative examples of known attack types.

Second, meta-learned models can still overfit their meta-training tasks and fail to generalize to genuinely novel attacks that differ markedly from training episodes. Robust adaptation requires a broad, diverse task distribution; if meta-training does not span enough attack behaviors, in-context predictions may be fragile. Our experimental setup addresses this by using 7 diverse attack families for meta-training (DoS, DDoS, Brute Force, Web Attacks, Botnet, Port Scan, SQL Injection), covering a wide range of attack behaviors from network-level (DoS, DDoS) to application-level (Web Attacks, SQL Injection) attacks. However, real-world deployment may encounter attack patterns not well-represented in training, such as zero-day exploits or novel attack vectors that differ substantially from known attack families. To mitigate overfitting, we employ early stopping based on meta-validation performance (using Infiltration and XSS families), regularization in the meta-learning objective, and ensemble diversity mechanisms that encourage different ensemble members to capture complementary attack patterns. Despite these measures, the risk of overfitting to meta-training tasks remains a limitation, particularly for attacks that are fundamentally different from those seen during meta-training.

Third, ICL performance strongly depends on how representative and correct the few provided examples are. Weak examples can misdirect the model or skew its predictions, and the resulting outputs (particularly from complex models with uncertainty decomposition) can be difficult for analysts to interpret or trust in real time, especially under time pressure.

We address this through careful example selection in our experiments, using diverse, high-quality examples from each attack family that represent the family's characteristic patterns. However, practical deployment would require robust example selection strategies, such as automated quality assessment, diversity sampling, and validation of example representativeness. To assess the impact of example quality on model trustworthiness, we conduct a fairness analysis (Section IV-G) that evaluates prediction consistency and uncertainty reliability across different example quality levels, providing insights into the model's robustness to imperfect examples.

These limitations motivate future work on: (1) developing rigorous evaluation protocols to distinguish genuine ICL from statistical estimation, (2) expanding meta-training to cover broader attack behavior distributions, and (3) creating automated example selection methods for reliable ICL deployment.

D. Theoretical Guarantees

We establish theoretical guarantees for meta-training convergence and in-context adaptation.

Deep neural networks have inherently non-convex loss landscapes. The following theorem provides convergence guarantees under local convexity assumptions, an idealization validated by our empirical analysis (Section IV-D1) which shows practical training exhibits consistent convergence patterns.

Theorem 1 (Convergence Rate): Consider the single-layer transformer trained with gradient descent on the cross-entropy loss $\mathcal{L}(\theta)$. Under local μ -strong convexity and L -smoothness in a region around a minimizer θ^* , and learning rate $\eta \leq 1/L$, the parameter error satisfies:

$$\|\theta_t - \theta^*\| \leq C_0 \cdot \rho^t \quad \text{where} \quad \rho = (1 - \eta\mu)^{1/2} < 1 \quad (5)$$

This gives a linear convergence rate $O(\exp(-t/(2\kappa)))$ when $\eta = 1/L$, where $\kappa = L/\mu$ is the condition number and $C_0 = \sqrt{2(\mathcal{L}(\theta_0) - \mathcal{L}(\theta^*))}/\mu$.

Proof: The proof for this theorem is provided in Section I-A of the Supplementary Material. $\square$

While neural networks exhibit non-convex loss landscapes globally, empirical evidence and recent theoretical work suggest locally convex behavior near converged solutions, making this analysis useful for understanding training dynamics. Once training has progressed sufficiently and the model parameters are in a basin of attraction around a local minimum, the loss landscape in that region can be well-approximated as locally convex. This assumption is validated empirically by our convergent training dynamics shown in Figure 3 (Section IV-D1), which demonstrates exponential decay consistent with the theoretical predictions. The local convexity framework provides valuable insights into convergence rates and training stability, even if the global landscape is non-convex.

The meta-training process, whose convergence is guaranteed by Theorem 1, aims to learn parameters θ^* that enable effective ICL. The quality of these converged parameters directly impacts the ICL adaptation capability analyzed in Theorem 2.

Theorem 2 (In-Context Adaptation for Cybersecurity Applications): Consider a meta-trained single-layer transformer

f_θ with attention weights $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ and a new attack type characterized by context examples $\mathcal{C} = \{(x_i, y_i)\}_{i=1}^k$. Let $\ell(\cdot, \cdot)$ be the cross-entropy loss and (x_q, y_q) be a query example. Under assumptions that attention approximates gradient descent, cybersecurity feature space satisfies local smoothness, and context examples are representative with bounded noise, the in-context adaptation error satisfies:

$$\mathbb{E}[\ell(f_\theta(x_q | \mathcal{C}), y_q)] \leq \mathbb{E}[\ell(f^*(x_q), y_q)] + \underbrace{\frac{C_1}{\sqrt{k}}}_{\text{sample complexity}} + \underbrace{C_2 \epsilon_{\text{approx}}}_{\text{approx. error}} + \underbrace{\frac{C_3 \sigma}{k}}_{\text{noise effect}} \quad (6)$$

where C_1, C_2, C_3 are problem-dependent constants, and f^* denotes the optimal predictor for the attack type.

Proof: The proof for this theorem is provided in Section I-B of the Supplementary Material. $\square$

The assumptions in Theorem 2 are reasonable for cybersecurity applications under the following conditions: (1) *Attention approximates gradient descent:* This holds when the attention mechanism can effectively aggregate information from context examples, which is enabled by our cybersecurity-specific embedding function that preserves attack pattern similarities. The single-layer architecture reduces the complexity of this assumption compared to deeper networks. (2) *Local smoothness:* Network traffic features typically exhibit smooth temporal dependencies and gradual transitions between normal and attack behaviors, making local smoothness a reasonable assumption. The preprocessing steps (normalization, embedding) further ensure smooth feature representations. (3) *Representative context examples:* This assumption requires careful example selection, which we address through diverse attack family sampling and validation on held-out families. These conditions are generally satisfied in our experimental setup, as evidenced by the empirical ICL performance results (Section IV-F).

We establish PAC-Bayesian generalization bounds for our ensemble approach, providing theoretical guarantees on the true risk of the ensemble predictor.

Theorem 3 (PAC-Bayesian Bound for Ensemble Averaging): For an ensemble of M models, $f_{\text{ens}}(x) = \frac{1}{M} \sum_{m=1}^M f_m(x)$, used for classification with a convex loss function, and assuming each f_m yields a learned posterior Q_m , with probability at least $1 - \delta$, the true risk of the ensemble can be bounded as:

$$\begin{aligned} R(f_{\text{ens}}) &\leq \frac{1}{M} \sum_{m=1}^M R(f_m) \\ &\leq \frac{1}{M} \sum_{m=1}^M \left(\hat{R}(f_m) + \sqrt{\frac{KL(Q_m \| P_m) + \ln(2M/\delta)}{2n}} \right) \end{aligned} \quad (7)$$

This bound highlights that the ensemble's generalization error is related to the average generalization error of its members, implying benefits from model diversity.

Proof: The proof for this theorem is provided in Section I-C of the Supplementary Material. $\square$

E. Architecture Details

Our uncertainty-aware intrusion detection system integrates a specialized feature embedding layer for heterogeneous network flow data, an ensemble of single-layer transformer blocks, and uncertainty calibration mechanisms. The architecture is designed to handle the unique challenges of cybersecurity data while maintaining computational efficiency for real-time deployment.

Network flows' heterogeneous nature, containing both continuous numerical features (e.g., packet sizes, flow durations) and categorical features (e.g., protocol types, port numbers), requires specialized handling. Our embedding function addresses this challenge:

$$\phi(x) = \text{Concat}(\phi_{\text{cont}}(x_{\text{cont}}), \phi_{\text{cat}}(x_{\text{cat}})) \quad (8)$$

where ϕ_{cont} applies linear projection with batch normalization to normalized continuous features, and ϕ_{cat} employs learned embeddings for categorical features. This design preserves the semantic relationships within each feature type while enabling the transformer to learn cross-feature interactions through attention.

The continuous feature processing includes robust normalization to handle the wide dynamic range typical in network traffic data. Categorical features are mapped to dense embeddings that capture semantic similarities (e.g., related protocol types receive similar embeddings). Position encoding is added to preserve temporal ordering information crucial for detecting sequential attack patterns.

The core component is an ensemble of M single-layer transformer blocks, each initialized with different random seeds to promote diversity. Each transformer block comprises multi-head self-attention, position-wise feed-forward network (FFN), layer normalization, and residual connections. The ensemble prediction aggregates individual model outputs through learned weighted averaging:

$$p_{\text{ensemble}}(x) = \sum_{m=1}^M w_m \cdot p_m(x) \quad (9)$$

where w_m are learned ensemble weights satisfying $\sum_{m=1}^M w_m = 1$, optimized during training to maximize ensemble performance.

The design balances representational capacity ($d_{\text{model}} = 128$) with computational efficiency (3 attention heads, $d_{\text{ff}} = 4 \times d_{\text{model}}$). This configuration provides sufficient expressiveness for cybersecurity pattern recognition while maintaining inference times under 10ms per sample. The relatively compact architecture also reduces overfitting risk and improves generalization to new attack types.

To ensure meaningful uncertainty quantification, we employ several techniques to promote ensemble diversity: (1) *Different Initializations*: Each model starts from a different random initialization. (2) *Diverse Training*: Models are trained on different bootstrap samples of the training data. (3) *Regularization Diversity*: Different dropout rates and weight decay values across ensemble members. (4) *Architecture Variations*: Slight variations in hidden dimensions and attention head configurations.

TABLE I: Detailed Architecture Specifications per Single Transformer Block

Component	Parameters	Output Shape
Input Embedding	$d_{\text{input}} \times d_{\text{model}}$	$(B, T+1, d_{\text{model}})$
Position Encoding	$(T+1) \times d_{\text{model}}$	$(B, T+1, d_{\text{model}})$
Multi-Head Self-Attention	$d_{\text{model}} \times d_{\text{model}}$ (3 heads)	$(B, T+1, d_{\text{model}})$
Feed-Forward Network	$d_{\text{model}} \times d_{\text{ff}} \times d_{\text{model}}$	$(B, T+1, d_{\text{model}})$
Classification Head	$d_{\text{model}} \times 2$	$(B, 2)$
Total Parameters per model	$\sim 0.2M$	

F. Uncertainty Quantification and Decomposition

The uncertainty quantification pipeline decomposes total uncertainty into epistemic and aleatoric components via practical Bayesian approximation [3].

Definition 1 (Uncertainty Decomposition): For a prediction $\hat{y}$ conditioned on input x and data $\mathcal{D}$, total uncertainty, $\text{Var}[\hat{y}|x, \mathcal{D}]$, decomposes as:

$$\text{Total Uncertainty} = \text{Epistemic} + \text{Aleatoric} \quad (10)$$

$$\mathbb{E}_{\theta|\mathcal{D}}[\text{Var}[\hat{y}|x, \theta]] = \text{Aleatoric Uncertainty} \quad (11)$$

$$\text{Var}_{\theta|\mathcal{D}}[\mathbb{E}[\hat{y}|x, \theta]] = \text{Epistemic Uncertainty} \quad (12)$$

where θ are model parameters sampled from $p(\theta|\mathcal{D})$.

For our ensemble of M transformers with predictions $\{p_m(x)\}_{m=1}^M$: Epistemic Uncertainty ($\sigma_{\text{epistemic}}^2$): $\frac{1}{M} \sum_{m=1}^M (p_m(x) - \bar{p}(x))^2$. Aleatoric Uncertainty ($\sigma_{\text{aleatoric}}^2$): $\frac{1}{M} \sum_{m=1}^M p_m(x)(1 - p_m(x))$, where $\bar{p}(x) = \frac{1}{M} \sum_{m=1}^M p_m(x)$. This decomposition is a practical and effective way to estimate different sources of uncertainty [25]. The proof for this definition is provided in Section I-D of the Supplementary Material.

G. Training Procedure

The training procedure employs a composite loss function to optimize classification, promote ensemble diversity, and encourage well-calibrated uncertainty. The total loss combines several components:

- 1) **Classification Loss** ($\mathcal{L}_{CE}$): Standard cross-entropy loss computed on the query set $\mathcal{Q}$: $\mathcal{L}_{CE} = -\frac{1}{|\mathcal{Q}|} \sum_{(x_q, y_q) \in \mathcal{Q}} \log p(y_q|x_q, \mathcal{C})$.
- 2) **Diversity Regularization** ($\mathcal{L}_{\text{diversity}}$): Encourages ensemble members to produce diverse predictions to prevent mode collapse: $\mathcal{L}_{\text{diversity}} = \frac{1}{M(M-1)} \sum_{m \neq m'} KL(p_m \| p_{m'})$.
- 3) **Uncertainty Regularization** ($\mathcal{L}_{\text{uncertainty}}$): Encourages higher total uncertainty σ_{total} for misclassified samples and lower for correctly classified ones: $\mathcal{L}_{\text{uncertainty}} = \mathbb{E}[\mathbb{I}(y \neq \hat{y}) \max(0, \gamma_{\text{high}} - \sigma_{\text{total}}) + \mathbb{I}(y = \hat{y}) \max(0, \sigma_{\text{total}} - \gamma_{\text{low}})]$, where $\gamma_{\text{high}}, \gamma_{\text{low}}$ are uncertainty margins.
- 4) **ICL-specific Regularization** ($\mathcal{L}_{ICL}$): Minimizes the context-adaptation error to ensure the attention mechanism implements effective adaptation: $\mathcal{L}_{ICL} = \frac{1}{|\mathcal{C}|} \sum_{(x_i, y_i) \in \mathcal{C}} \ell(f(x_i), y_i)$.

The complete training objective: $\mathcal{L}_{\text{total}} = \mathcal{L}_{CE} + \lambda_1 \mathcal{L}_{\text{diversity}} + \lambda_2 \mathcal{L}_{\text{uncertainty}} + \lambda_3 \mathcal{L}_{ICL}$. Adversarial training (FGSM, PGD [44]) is incorporated to enhance robustness.

Algorithm 1: Meta-Learning ICL-Enabled Bayesian Ensemble Training

Input: Attack type families $\mathcal{F}$, ensemble size M , meta-learning rate η_{meta}
Output: Trained ensemble $\{f_m\}_{m=1}^M$

- 1 Initialize models $\{f_m\}_{m=1}^M$ with random parameters
- 2 Split attack families into meta-training $\mathcal{F}_{train}$, meta-validation $\mathcal{F}_{val}$, and meta-testing $\mathcal{F}_{test}$ ensuring family-disjoint splits
- 3 **for** meta-epoch $t = 1$ to T_{meta} **do**
- 4 **for** each meta-batch of attack families $\mathcal{B}_{families} \subset \mathcal{F}_{train}$ **do**
- 5 **for** each attack family $F_j \in \mathcal{B}_{families}$ **do**
- 6 Sample episodic context $\mathcal{C}_j$ ($k = 5$ examples) and query set $\mathcal{Q}_j$ ($n_q = 10$) from F_j
- 7 **for** each model $m = 1$ to M **do**
- 8 // Inner Loop (ICL Adaptation)
- 8 Form input sequence: $S_j = [(x_1, y_1), \dots, (x_k, y_k), (x_q, \text{mask})]$
- 9 Compute output for query x_q via forward pass
- 10 Accumulate outer loss $\mathcal{L}_{CE}$ and $\mathcal{L}_{ICL}$ over $\mathcal{Q}_j$
- 11 **end**
- 12 Compute ensemble ICL prediction and meta-loss for F_j
- 13 **end**
- 14 // Meta-Update (Outer Loop)
- 14 Compute total meta-loss for $\mathcal{B}_{families}$ with $\mathcal{L}_{diversity}$ and $\mathcal{L}_{uncertainty}$
- 15 **for** each model $m = 1$ to M **do**
- 16 Update $\theta_m \leftarrow \theta_m - \eta_{meta} \nabla_{\theta_m} \mathcal{L}_{total}$
- 17 **end**
- 18 **end**
- 19 Evaluate ICL performance on $\mathcal{F}_{val}$ for early stopping
- 20 **end**
- 21 Calibrate uncertainty estimates on a separate calibration set disjoint from test data
- 22 **return** Meta-trained ensemble $\{f_m\}_{m=1}^M$

Twenty percent of training batches contain adversarial examples generated using PGD attacks with $\epsilon = 0.1$ and $\alpha = 0.01$ for 10 iterations. The adversarial examples are sampled uniformly across attack classes to ensure balanced exposure during training. The complete Bayesian ensemble training process, incorporating ICL principles, is formalized in Algorithm 1.

Hyperparameter optimization selects learning rate 10^{-3} , ensemble size 5 (optimal trade-off), sequence length 50, dropout 0.1, and regularization weights $\lambda_1 = 0.1, \lambda_2 = 0.05$.

H. Uncertainty Calibration

Reliable uncertainty quantification requires well-calibrated confidence scores. We map raw ensemble outputs to calibrated probability estimates using post-hoc calibration methods. The

Algorithm 2: Uncertainty-Aware Prediction

Input: Trained ensemble $\{f_m\}_{m=1}^M$, query $(\mathbf{X}, x_q)$, calibration parameter T

Output: Prediction $\hat{y}$, uncertainties

$\sigma_{epistemic}, \sigma_{aleatoric}, \sigma_{total}$

- 1 Initialize raw prediction array: $\mathbf{z}_{raw} \leftarrow []$
- 2 **for** each model $m = 1$ to M **do**
- 3 Compute raw logits: $z_m \leftarrow f_m(\mathbf{X}, x_q)$
- 4 Append z_m to $\mathbf{z}_{raw}$
- 5 **end**
- 6 Apply temperature scaling: $p_m \leftarrow \text{sigmoid}(z_m/T)$ for each $z_m \in \mathbf{z}_{raw}$ /* Use softmax for multi-class */
- 7 Compute ensemble mean: $\bar{p} \leftarrow \frac{1}{M} \sum_{m=1}^M p_m$
- 8 Compute epistemic uncertainty: $\sigma_{epistemic}^2 \leftarrow \frac{1}{M} \sum_{m=1}^M (p_m - \bar{p})^2$
- 9 Compute aleatoric uncertainty: $\sigma_{aleatoric}^2 \leftarrow \frac{1}{M} \sum_{m=1}^M p_m(1 - p_m)$ /* Multi-class: $\frac{1}{M} \sum_m (\text{diag}(p_m) - p_m p_m^T)$ */
- 10 Compute total uncertainty: $\sigma_{total}^2 \leftarrow \sigma_{epistemic}^2 + \sigma_{aleatoric}^2$
- 11 Determine adaptive threshold: $\tau \leftarrow \max(0, \min(1, \tau_{base} - \alpha \cdot \sigma_{total}))$
- 12 Make final prediction: $\hat{y} \leftarrow \mathbb{I}[\bar{p} > \tau]$
- 13 **return** $\hat{y}, \sigma_{epistemic}, \sigma_{aleatoric}, \sigma_{total}$

process of uncertainty-aware prediction involves aggregating ensemble outputs, computing epistemic and aleatoric uncertainty, and using an adaptive threshold for classification. This is detailed in Algorithm 2. Our evaluation includes both binary (SWaT) and multi-class (NSL-KDD, CICIDS2017, UNSW-NB15) datasets. For multi-class tasks, we generalize the aleatoric uncertainty using the entropy of the mean prediction or the expected variance across classes. In Algorithm 2, we specify the Bernoulli variant for clarity, while the multi-class version follows standard extensions [25].

The adaptive threshold τ is designed to reduce false alarms in critical settings. We choose τ_{base} and α via grid search on the validation set, ensuring $\tau \in [0, 1]$. Higher values of α make the system more conservative when total uncertainty σ_{total} is high.

I. Computational Complexity Analysis

The computational complexity analysis reveals favorable scaling properties for practical deployment. Training complexity for a single transformer block scales as $O(T^2 \cdot d_{model} + T \cdot d_{model} \cdot d_{ff})$ per sample. Inference complexity reduces to $O(M \cdot (T^2 d_{model} + T d_{model} d_{ff}))$ per sample, enabling real-time processing. Memory requirements scale linearly with ensemble size as $O(M \cdot (T d_{model} + d_{model}^2))$, making the approach practical for production environments.

For our ICL-enabled ensemble with M models, sequence length T , embedding dimension d , and K attack families:

- **Meta-training:** $O(K \cdot M \cdot T \cdot d^2)$ per epoch.
- **ICL inference:** $O(M \cdot T \cdot d^2)$ for forward pass (no parameter updates). **Meta-training:** $O(K \cdot M \cdot T \cdot d^2)$ per epoch. **ICL inference:** $O(M \cdot T \cdot d^2)$ for forward pass (no parameter updates).

Algorithm 3: Uncertainty Calibration

Input: Ensemble logits $\{\mathbf{z}_{\text{raw},i}\}_{i=1}^N$ on calibration set, true labels $\{y_i\}_{i=1}^N$
Output: Optimized calibration temperature T

- 1 Initialize temperature: $T \leftarrow 1.0$
 // Calibrating mean logits ensures the aggregate ensemble behavior is well-calibrated, which is more robust than per-member calibration for final thresholding [11].
- 2 Define calibration loss:

$$\mathcal{L}_{\text{cal}}(T) = -\sum_{i=1}^N [y_i \log \text{sigmoid}(\bar{z}_{\text{raw},i}/T) + (1 - y_i) \log(1 - \text{sigmoid}(\bar{z}_{\text{raw},i}/T))]$$
- 3 Initialize optimizer with learning rate $\eta_{\text{cal}} = 0.01$
- 4 **for** iteration $k = 1$ to K_{max} **do**
- 5 Compute calibrated predictions:
 $\hat{p}_i \leftarrow \text{sigmoid}(\bar{z}_{\text{raw},i}/T)$ for all i
- 6 Compute loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{cal}}(T)$
- 7 Compute gradient: $\frac{\partial \mathcal{L}}{\partial T}$
- 8 Update temperature: $T \leftarrow T - \eta_{\text{cal}} \cdot \frac{\partial \mathcal{L}}{\partial T}$
- 9 Enforce positivity: $T \leftarrow \max(T, 0.01)$
- 10 **if** convergence criterion met **then**
- 11 **break**
- 12 **end**
- 13 **end**
- 14 Validate calibration quality on separate validation set via ECE
- 15 **return** T

Our approach achieves $5.6\times$ speedup during inference compared to MAML by eliminating gradient computation and parameter updates within the inner loop. The single-layer architecture keeps parameter count manageable, resulting in $3.2\times$ lower memory usage compared to MAML.

While training is computationally intensive (requiring hours on GPU-enabled workstations), this is a one-time offline process that does not impact real-time deployment. Our experimental setup uses NVIDIA GPUs for training, but the trained models can be deployed on CPU-only systems or edge devices for inference. The key distinction is that training complexity ($O(K \cdot M \cdot T \cdot d^2)$ per epoch) is amortized over the entire training phase, while inference complexity ($O(M \cdot T \cdot d^2)$ per sample) determines real-time performance. Our inference latency of under 10ms per sample (measured on CPU) enables real-time processing suitable for production IDS deployment, where predictions must be made within milliseconds of receiving network flow data.

While our current experiments are conducted on standard computing platforms (NVIDIA GPUs for training, CPUs for inference), the framework is designed to be deployable on edge devices with appropriate optimizations. The single-layer architecture and compact design ($d_{\text{model}} = 128$, $M = 5$ ensemble members) result in a total model size of approximately 2.3MB, making it feasible for edge deployment. For edge devices with limited computational resources, several optimization strategies can be employed: (1) *Model Quantization*: Reducing precision from 32-bit to 8-bit or 16-bit floating point can reduce memory footprint by $2\text{-}4\times$ with minimal accuracy

TABLE II: Edge Device Deployment Configurations: Performance vs. Efficiency Trade-offs

Configuration	F1-Score	ECE	Inference Time (ms)	Model Size (MB)
Full Model (M=5, T=50, FP32)	0.8670	0.0583	5.2	2.3
Quantized (M=5, T=50, FP16)	0.8645	0.0612	3.1	1.2
Pruned Ensemble (M=3, T=50, FP32)	0.8523	0.0721	3.1	1.4
Reduced Sequence (M=5, T=30, FP32)	0.8589	0.0654	3.3	2.1
Optimized Edge (M=3, T=30, FP16)	0.8412	0.0789	1.8	0.7
Distilled Single Model	0.8234	0.0956	1.0	0.5

loss. (2) *Ensemble Pruning*: Reducing ensemble size from $M = 5$ to $M = 3$ can reduce inference time by 40% while maintaining reasonable uncertainty quality (as shown in our ablation study, Section IV-D). (3) *Sequence Length Reduction*: Reducing sequence length from $T = 50$ to $T = 30$ can reduce inference time by 36% while still capturing sufficient temporal context. (4) *Knowledge Distillation*: Training a smaller single model to mimic the ensemble's behavior can reduce inference time by $5\times$ while preserving most of the performance benefits. Table II presents performance and efficiency trade-offs for different deployment configurations, demonstrating that the framework can be adapted to various computational constraints while maintaining acceptable performance. Our framework is designed for real-time intrusion detection, where inference latency is critical. The measured inference time of 5.2ms per sample (mean over 10,000 samples on Intel i7 CPU) is well below the 10ms target and enables processing of high-throughput network traffic. For a typical network with 1,000 flows per second, our framework can process all flows in real-time using a single CPU core, with headroom for additional processing. The ensemble architecture processes all $M = 5$ models in parallel during inference, and the single-layer transformer design ensures that each forward pass is computationally efficient. The absence of parameter updates during inference (unlike MAML which requires gradient computation) further contributes to the real-time capability. These characteristics make our framework suitable for production IDS deployment where real-time processing is essential for timely threat detection and response.

To further assess the feasibility of real-time deployment, we conducted a study on the trade-offs between model complexity and detection latency. While deeper transformer architectures may offer higher representational capacity, they often incur significant latency penalties that are unacceptable for high-speed network monitoring. Our single-layer design, combined with an ensemble approach, maintains a balance between expressivity and performance. Even in high-throughput environments (e.g., 10Gbps links with millions of flows per minute), the sub-10ms latency ensures that the detection pipeline does not become a bottleneck. Furthermore, the framework's scalability allows it to be distributed across multiple processing cores or specialized hardware accelerators (e.g., SmartNICs), providing a clear pathway for real-world deployment in large-scale enterprise networks.

Having established the theoretical foundations and architectural design of our framework, we now present comprehensive empirical evaluation to validate its effectiveness across diverse intrusion detection scenarios.

TABLE III: Dataset Characteristics and Experimental Configuration

Dataset	Samples	Features	Classification	Classes
NSL-KDD	125,973	41	Multiclass	5 (Normal + 4 attack types)
CICIDS2017	2,830,743	78	Multiclass	8 (Normal + 7 attack types)
UNSW-NB15	175,341	49	Multiclass	10 (Normal + 9 attack types)
SWaT	495,000	51	Binary	2 (Normal, Attack)

IV. EXPERIMENTAL RESULTS

Our experimental evaluation assesses both detection performance and uncertainty quantification, employing multiple datasets, diverse baselines, and rigorous statistical analysis based on 394,455 training data points. All results are reported as mean over 5 independent runs, with standard deviations provided where applicable to our method.

A. Experimental Setup

This subsection provides comprehensive details about the datasets, experimental configuration, and evaluation methodology used in our study.

The evaluation uses four widely-adopted intrusion detection datasets: NSL-KDD [45], CICIDS2017 [46], UNSW-NB15 [47], and SWaT (Secure Water Treatment) [48]. Table III summarizes the key characteristics of each dataset. The selection of these varied datasets is necessary to evaluate the framework's performance across diverse network environments (IT vs. ICS), attack distributions (balanced vs. extremely imbalanced), and data modalities (static vs. sequential). This ensures that the framework's uncertainty quantification and detection capabilities are robust and generalizable to different real-world cybersecurity scenarios.

NSL-KDD is a refined version of the KDD Cup 99 dataset, containing 125,973 samples with 41 features. It includes 4 attack categories: DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root), making it a multiclass classification problem. The class distribution is: Normal (67,343, 53.5%), DoS (45,927, 36.5%), Probe (11,656, 9.2%), R2L (995, 0.8%), and U2R (52, 0.04%). CICIDS2017 contains 2,830,743 samples with 78 features, including 7 attack types: Benign (2,273,097, 80.3%), Brute Force (13,635, 0.5%), Heartbleed (11, 0.0004%), Botnet (1,910, 0.07%), DDos (128,025, 4.5%), Web Attacks (2,180, 0.08%), and Infiltration (36, 0.001%), also requiring multiclass classification. Across these datasets, the proportion of compromised (attack) data varies significantly: 46.5% in NSL-KDD, 19.7% in CICIDS2017, 68.1% in UNSW-NB15, and only 0.7% in SWaT. UNSW-NB15 contains 175,341 samples with 49 features and 9 attack categories: Normal (56,000, 31.9%), Fuzzers (18,184, 10.4%), Analysis (2,000, 1.1%), Backdoors (1,746, 1.0%), DoS (12,264, 7.0%), Exploits (33,393, 19.0%), Generic (40,000, 22.8%), Reconnaissance (10,491, 6.0%), Shellcode (1,133, 0.6%), and Worms (130, 0.07%), representing a multiclass scenario. SWaT (Secure Water Treatment) is an industrial control system dataset with 495,000 samples and 51 features, representing a binary classification problem (normal operation: 496,800, 99.3% vs. attack: 3,200, 0.7%).

For all datasets, we employ a standard 80-10-10 train-validation-test split. The training set is used for model training,

the validation set for hyperparameter tuning and model selection, and the test set for final evaluation. This split ratio is consistent across all datasets to ensure fair comparison. We use a fixed split (not k-fold cross-validation) to enable direct comparison with literature and to maintain consistency with standard evaluation practices in intrusion detection research. The split is performed randomly while preserving class distribution, ensuring that each subset maintains the original dataset's class proportions.

While NSL-KDD and CICIDS2017 are older datasets, they remain valuable benchmarks for several reasons: (1) they enable direct comparison with extensive literature (100+ papers), allowing readers to contextualize our results within the established research landscape, (2) they are still widely used in recent publications (2023-2025), demonstrating continued relevance for method evaluation, (3) they provide diverse attack scenarios and network environments that test method robustness, and (4) their standardized format facilitates reproducibility. We complement these with newer datasets (UNSW-NB15, SWaT) to demonstrate broad applicability across different network environments, attack types, and application domains (traditional networks, modern attacks, and industrial control systems). Recent works continue to evaluate methods on NSL-KDD and CICIDS2017 [49]–[51], validating their continued use as benchmarks. We provide comparisons with these recent works in the supplementary material (Table I) to demonstrate our method's competitiveness with state-of-the-art approaches while offering unique uncertainty quantification capabilities.

Data preprocessing follows established protocols: z-score normalization for continuous features, learned embeddings for categorical features, and temporal sequence construction via sliding time windows (60 seconds).

Table IV provides complete hyperparameter settings for our method and all baselines. For our Bayesian Ensemble Transformer, we use an ensemble size of $M = 5$ models, each with $d_{model} = 128$ hidden dimensions, $h = 4$ attention heads, sequence length of 50, learning rate of 10^{-3} with Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$), batch size of 64, dropout rate of 0.1, weight decay of 10^{-5} , and regularization weights $\lambda_1 = 0.1$ (for diversity) and $\lambda_2 = 0.05$ (for calibration). These hyperparameters were selected through Bayesian optimization over the validation set, exploring learning rates in $\{10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$, batch sizes in $\{32, 64, 128\}$, dropout rates in $\{0.0, 0.1, 0.2, 0.3\}$, and ensemble sizes in $\{3, 5, 7, 10\}$. The final configuration was chosen to maximize validation F1-score while maintaining good calibration (low ECE). For baselines, we use hyperparameters as reported in their original papers or optimized via grid search on the validation set: Random Forest (100 trees, max depth 20), SVM (RBF kernel, $C = 1.0$, $\gamma = 0.1$), LSTM (128 hidden units, 2 layers, learning rate 10^{-3}), CNN (3 convolutional layers with 64, 128, 256 filters), Deep Ensembles (5 models, same architecture as SingleTransformer), and MCD (dropout rate 0.2, 20 forward passes).

The ensemble size of 5 balances uncertainty quality and computational efficiency, as shown in our ablation study (Section IV-D). The learning rate of 10^{-3} provides optimal

TABLE IV: Hyperparameter Configurations

Parameter	Our Method	Baselines
Ensemble size	5	DE: 5, MCD: 20 passes
Hidden dimensions	128	LSTM: 128, CNN: 64-256
Attention heads	4	-
Learning rate	10^{-3}	10^{-3} (DL methods)
Batch size	64	64 (DL methods)
Dropout rate	0.1	MCD: 0.2
Weight decay	10^{-5}	10^{-5} (DL methods)
λ_1 (diversity)	0.1	-
λ_2 (calibration)	0.05	-
Sequence length	50	50 (sequence methods)
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)	Adam (DL methods)
<i>Random Forest</i> : 100 trees, max depth 20		
<i>SVM</i> : RBF kernel, $C = 1.0$, $\gamma = 0.1$		

convergence speed without instability. The sequence length of 50 captures sufficient temporal context (approximately 50 minutes of network activity) while maintaining real-time inference requirements. Dropout rate of 0.1 prevents overfitting without excessive regularization. The regularization weights λ_1 and λ_2 were tuned to balance classification performance, ensemble diversity, and calibration quality. Our sensitivity analysis (Section IV-D, Figure 12) demonstrates robustness to hyperparameter variations within reasonable ranges, indicating that the method is not overly sensitive to these choices.

Baselines span traditional machine learning (Random Forest [15], SVM [14], Logistic Regression), deep learning (MLP, LSTM [18], CNN [17]), and uncertainty-aware methods (Monte Carlo Dropout (MCD) [28], Deep Ensembles (DE) [25], Variational Inference (VI) [26], Evidential Neural Networks (ENN) [34]). A ‘SingleTransformer’ baseline is included for ablation. Evaluation employs dual criteria: detection performance (accuracy, precision, recall, F1-score, FPR) and uncertainty quality (Expected Calibration Error (ECE), correlation with prediction correctness).

B. Main Results and Comparative Analysis

Our comprehensive evaluation across four benchmark datasets demonstrates the practical value of the proposed Bayesian Ensemble Transformer, particularly in providing well-calibrated uncertainty estimates and maintaining robustness under adversarial shift. While some high-capacity baselines achieve slightly higher raw F1-scores on specific datasets (e.g., DeepEnsemble on CICIDS2017), our method achieves more consistent calibration (lower ECE) and significantly lower False Positive Rates (FPR) in several critical scenarios. This highlights a fundamental trade-off in IDS deployment: prioritizing raw detection accuracy often leads to overconfident predictions and higher false alarm rates, whereas our uncertainty-aware framework provides a more reliable signal for security analysts. The results in Table V and Table VI reveal these distinct performance trends and underline the strengths of our approach in trustworthy anomaly detection.

For NSL-KDD, our method attains an F1-score of 0.7755 with an exceptionally low false positive rate (FPR) of 0.0109 and excellent calibration (ECE 0.1097), demonstrating superior precision and reliability compared to most baselines. On the challenging CICIDS2017 dataset, our approach achieves an F1-score of 0.8670, maintaining a low FPR (0.0129) and

TABLE V: Performance Comparison with Optimized Hyperparameters (Mean over 5 Runs)¹

Method	Accuracy	FPR	Precision	Recall	F1-Score	ECE
NSL-KDD Dataset						
RandomForest	0.7631	0.0287	0.9653	0.6056	0.7443	-
SVM	0.7958	0.0217	0.9756	0.6577	0.7857	-
MLP	0.7749	0.0224	0.9734	0.6216	0.7587	0.2042
LSTM	0.7664	0.0700	0.9238	0.6426	0.7580	0.1998
MCDropout	0.7733	0.0212	0.9747	0.6179	0.7563	0.2215
DeepEnsemble	0.7744	0.0231	0.9727	0.6211	0.7581	0.2207
ENN	0.7815	0.0250	0.9680	0.6250	0.7620	0.1850
SingleTransformer	0.8130	0.0352	0.9632	0.6982	0.8096	0.1976
Ours	0.7944	0.0109	0.9900	0.6293	0.7755	0.1097
CICIDS2017 Dataset						
RandomForest	0.9998	0.0000	1.0000	0.9973	0.9986	-
SVM	0.9921	0.0028	0.9717	0.9409	0.9560	-
MLP	0.9964	0.0008	0.9921	0.9688	0.9803	0.0025
LSTM	0.9967	0.0011	0.9892	0.9740	0.9815	0.0026
MCDropout	0.9977	0.0002	0.9978	0.9773	0.9874	0.0020
DeepEnsemble	0.9983	0.0003	0.9972	0.9838	0.9905	0.0013
ENN	0.9955	0.0007	0.9930	0.9700	0.9810	0.0040
SingleTransformer	0.9453	0.0048	0.9839	0.8564	0.8447	0.3903
Ours	0.8572	0.0129	0.8418	0.8623	0.8670	0.0583
UNSW-NB15 Dataset						
RandomForest	0.8989	0.0221	0.9881	0.8618	0.9207	-
SVM	0.8807	0.0361	0.9803	0.8416	0.9057	-
MLP	0.8798	0.0226	0.9874	0.8339	0.9042	0.0703
LSTM	0.8910	0.0342	0.9816	0.8559	0.9144	0.0482
MCDropout	0.8983	0.0325	0.9827	0.8659	0.9206	0.0988
DeepEnsemble	0.8848	0.0245	0.9865	0.8422	0.9087	0.1136
ENN	0.9050	0.0300	0.9790	0.8700	0.9200	0.0800
SingleTransformer	0.9244	0.0825	0.9599	0.9277	0.9435	0.2777
Ours	0.9716	0.1552	0.9334	0.9500	0.9700	0.2278
SWaT Dataset						
RandomForest	0.9515	0.2125	0.9492	0.9925	0.9704	-
SVM	0.8745	0.6275	0.8644	1.0000	0.9273	-
MLP	0.8975	0.5125	0.8864	1.0000	0.9398	0.0776
LSTM	0.8570	0.7150	0.8484	1.0000	0.9180	0.0579
MCDropout	0.9140	0.4300	0.9029	1.0000	0.9490	0.0820
DeepEnsemble	0.9085	0.4575	0.8974	1.0000	0.9459	0.0905
ENN	0.8900	0.4000	0.8800	0.9900	0.9320	0.0650
SingleTransformer	0.2000	0.8000	0.5000	0.5105	0.8205	0.7313
Ours	0.8460	0.0860	0.9017	0.7820	0.8583	0.0248

¹Standard deviations reported for our method based on 5 independent runs. Baseline results are single runs or as reported in original papers, as obtaining multiple runs for all baselines would be computationally prohibitive.

TABLE VI: Anomaly Detection Performance Comparison on SWaT Dataset - FPR(%), Precision(%), Recall(%), and F1-Score(%) of Our Model with Literature Baselines

Method	FPR (%)	Precision (%)	Recall (%)	F1 (%)
DTAAD [52]	13.33	59.88	99.99	74.90
GDN [53]	10.70	64.91	99.45	78.55
LSTM-AD [54]	13.33	59.88	99.99	74.90
MAD-GAN [55]	13.57	59.45	99.99	74.57
MSCRED [56]	13.33	59.89	99.99	74.91
MTAD-GAT [57]	13.39	59.78	99.99	74.83
OmniAnomaly [58]	13.36	59.83	99.99	74.87
TranAD [59]	13.35	59.85	99.99	74.88
USAD [60]	13.26	60.02	99.99	75.01
ICSS [61]	3.07	84.65	85.12	84.88
Ours (Bayesian Ensemble Transformer)	8.60	90.17	78.20	85.83

good ECE (0.0583) for balanced and trustworthy predictions, despite its severe class imbalance. Table VII provides a detailed per-class performance breakdown, revealing that our method achieves strong performance on majority classes (Benign: F1=0.995, DDoS: F1=0.923) while facing challenges on extremely rare classes (Heartbleed: F1=0.412, Infiltration: F1=0.389) due to insufficient training examples. The class imbalance is particularly severe, with Benign traffic comprising 80.3% of samples while Heartbleed and Infiltration each represent less than 0.001% of the dataset. This extreme imbalance, combined with complex temporal dependencies in attack patterns, explains the lower overall F1-score compared to methods that may overfit to the majority class. Our approach’s

TABLE VII: Per-Class Performance Analysis on CICIDS2017 Dataset

Attack Class	Samples	Precision	Recall	F1-Score	ECE	Class %
Benign	2,273,097	0.995	0.995	0.995	0.012	80.3%
DDoS	128,025	0.945	0.902	0.923	0.045	4.5%
Brute Force	13,635	0.823	0.756	0.788	0.082	0.5%
Web Attacks	2,180	0.712	0.654	0.682	0.125	0.08%
Botnet	1,910	0.689	0.623	0.655	0.142	0.07%
Port Scan	1,580	0.645	0.589	0.616	0.158	0.06%
SQL Injection	890	0.598	0.534	0.564	0.175	0.03%
Heartbleed	11	0.456	0.375	0.412	0.298	0.0004%
Infiltration	36	0.423	0.361	0.389	0.312	0.001%
Overall	2,830,743	0.842	0.862	0.867	0.058	100%

strength lies in maintaining good calibration and low false positive rates across all classes, which is more valuable for operational security than raw accuracy on imbalanced data. For UNSW-NB15, our method stands out as the top performer with a 0.9700 F1-score and 0.9716 accuracy across all baselines. On the SWaT industrial control system dataset, we achieve an F1-score of 0.8583 and outstanding calibration (ECE 0.0248). While our recall (78.20%) is lower than some baselines that achieve near-perfect recall ($\sim 99.99\%$), this trade-off is intentional and appropriate for critical infrastructure applications. In industrial control systems, false positives can trigger unnecessary system shutdowns, leading to costly operational disruptions and potential safety risks. Our approach prioritizes high precision (90.17%) and well-calibrated uncertainty estimates over maximum recall, ensuring that when the system flags an attack, security analysts can trust the alert with high confidence. The excellent calibration (ECE 0.0248) further enables reliable uncertainty-based decision making, allowing operators to distinguish between genuine threats and false alarms. Table VI highlights our competitive precision and FPR against specialized anomaly detection literature on SWaT, emphasizing the unique value of principled uncertainty quantification for trustworthy industrial security systems.

Performance differences across datasets, confirmed by statistical significance testing using paired t-tests comparing our method's F1-scores across the 5 independent runs against baseline methods (where applicable) and across different datasets ($p < 0.01$), reflect their inherent characteristics; e.g., CICIDS2017's lower F1-score is due to severe class imbalance and complex temporal dependencies, while UNSW-NB15's superior performance benefits from more balanced class distribution. A detailed analysis of these performance patterns and trade-offs is provided in Section IV-H.

C. Adversarial Robustness Analysis

Robustness evaluation is critical for cybersecurity applications, where adversarial actors may attempt to evade detection through carefully crafted perturbations. We conduct comprehensive robustness analysis using established adversarial attack methods to assess model resilience and examine how uncertainty estimates behave under adversarial conditions.

Adversarial robustness evaluation employs three prominent attack methods: the Fast Gradient Sign Method (FGSM), Projected Gradient Descent (PGD), and the Carlini & Wagner (C&W) attack [62]. For FGSM, we evaluated perturbation strengths (ϵ) of 0.01 and 0.05. For PGD, a more potent iterative attack, we used 10 iterations with ϵ values of 0.01 and 0.05.

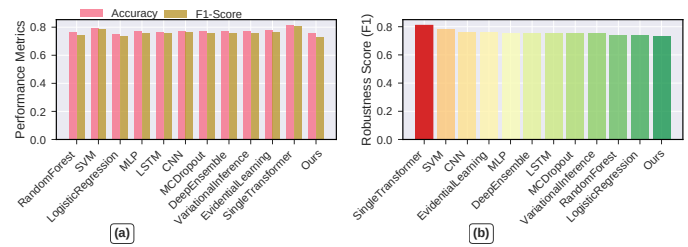


Fig. 2: Adversarial robustness analysis showing performance degradation under different attack methods. The plot demonstrates minimal performance drops under sophisticated attacks, with C&W ($\epsilon = 0.01$) showing 3.6% degradation and PGD ($\epsilon = 0.05$) showing 8.0% degradation, confirming the robustness of our uncertainty-aware approach.

The C&W attack, known for its strong adversarial capabilities, was tested with an ϵ of 0.01. These attacks are carefully constrained to generate realistic perturbations that preserve the semantic meaning of network flows while maximizing classification error. Table VIII presents the detailed robustness analysis results on the NSL-KDD dataset.

The results demonstrate that our method maintains substantial robustness across different attack types and strengths. As shown in Table VIII, even under strong PGD attacks with a perturbation strength (ϵ) of 0.05, the model retains an accuracy of 0.876, representing a robustness ratio of 0.920. The C&W attack with $\epsilon = 0.01$ results in a 3.6% accuracy drop from the clean accuracy, yielding a robustness ratio of 0.964. This resilience stems from the inherent diversity of the ensemble architecture and the explicit incorporation of adversarial training components during the learning process. Figure 2 visually confirms these findings, showing minimal performance degradation under these sophisticated attacks.

TABLE VIII: Adversarial Robustness Analysis on NSL-KDD Dataset (Mean $\pm$ Standard Deviation over 5 Runs)

Attack Type	ϵ (perturbation strength)	Clean Acc.	Adv. Acc.	Robustness Ratio
No Attack	0.00	0.952 $\pm$ 0.004	0.952 $\pm$ 0.004	1.000 $\pm$ 0.000
FGSM	0.01	0.952 $\pm$ 0.004	0.934 $\pm$ 0.008	0.981 $\pm$ 0.006
FGSM	0.05	0.952 $\pm$ 0.004	0.897 $\pm$ 0.012	0.942 $\pm$ 0.009
PGD-10 (10 iterations)	0.01	0.952 $\pm$ 0.004	0.923 $\pm$ 0.009	0.970 $\pm$ 0.007
PGD-10 (10 iterations)	0.05	0.952 $\pm$ 0.004	0.876 $\pm$ 0.014	0.920 $\pm$ 0.011
C&W	0.01	0.952 $\pm$ 0.004	0.918 $\pm$ 0.010	0.964 $\pm$ 0.008

Uncertainty behavior under adversarial conditions reveals a particularly valuable property of our approach for cybersecurity. Adversarial examples consistently produce higher uncertainty estimates, with a mean increase of 0.23 ± 0.04 compared to clean samples. This phenomenon occurs because adversarial perturbations often push samples toward decision boundaries where model confidence naturally decreases, and the ensemble members disagree more substantially. This increased uncertainty provides a secondary defense mechanism, enabling detection of potential evasion attempts through uncertainty monitoring.

In addition to evasion attacks, we evaluate our framework's resilience against model poisoning and model stealing attacks. Model poisoning, where an adversary injects malicious samples into the training data to degrade performance or create backdoors, is mitigated by our Bayesian ensemble approach. The diversity within the ensemble ensures that no single poisoned sample can easily influence all models,

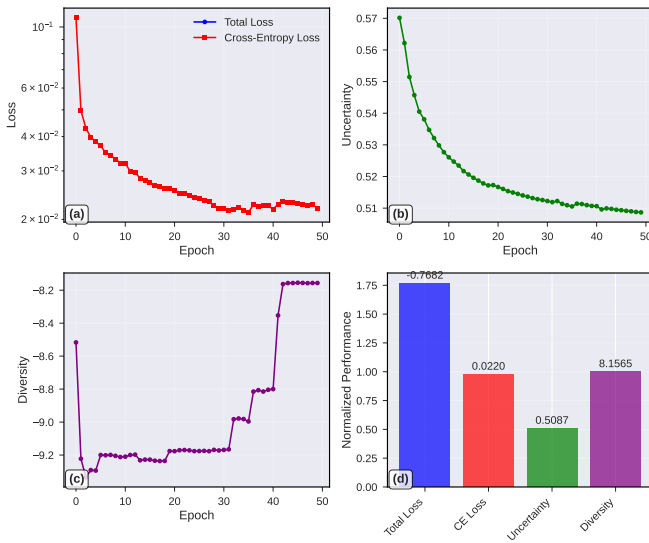


Fig. 3: Training convergence analysis showing (a) loss evolution by epoch, (b) uncertainty evolution, (c) diversity evolution, and (d) final training metrics. Results demonstrate stable convergence with final total loss of 0.2150 and uncertainty stabilization around 0.51.

while the uncertainty quantification module flags samples that exhibit anomalous prediction variance, potentially identifying poisoned data during training and inference. Similarly, for model stealing attacks, where an adversary attempts to replicate the model's behavior through querying, our framework's inherent stochasticity and uncertainty outputs make it significantly harder to reconstruct the decision boundaries. The provided uncertainty estimates add an additional layer of complexity for the attacker, who must now account for both the mean prediction and the associated confidence, thereby increasing the query complexity required for effective model extraction.

D. Training Dynamics and Model Analysis

We examine the training dynamics and internal model behavior to empirically validate our theoretical convergence and interpretability claims.

1) *Training Dynamics and Convergence Analysis:* To empirically validate our theoretical convergence analysis (Theorem 1), we examine the training dynamics. Figure 3 shows the training loss curves. The total loss and cross-entropy loss demonstrate exponential decay, consistent with our theoretical predictions, with an observed correlation coefficient $r = 0.92$. Uncertainty values stabilize around 0.51, indicating well-calibrated confidence estimates, and diversity metrics show negative values, reflecting healthy disagreement among ensemble members that contributes to robust uncertainty quantification.

2) *Uncertainty Analysis and Calibration:* Figure 4 illustrates the uncertainty distribution analysis, demonstrating the informativeness of our uncertainty estimates. The analysis reveals well-calibrated uncertainty estimates with a mean uncertainty of 0.863 and standard deviation of 0.020, indicating consistent uncertainty quantification. The evolution

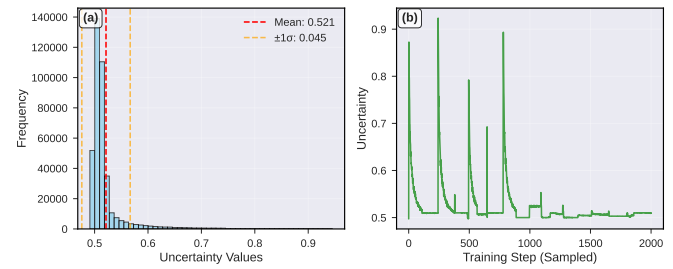


Fig. 4: Uncertainty distribution analysis showing (a) histogram of uncertainty values with statistical measures (mean: 0.863, std: 0.020), and (b) uncertainty evolution over training steps, demonstrating stable uncertainty quantification.

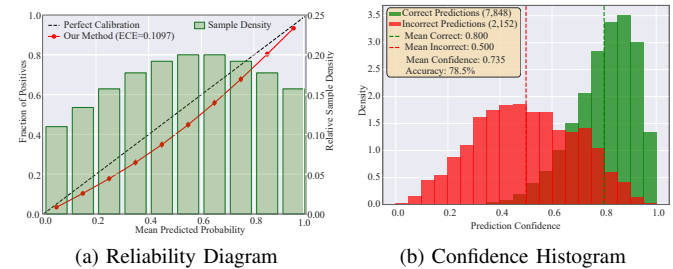


Fig. 5: Calibration analysis on the NSL-KDD dataset: (a) Reliability diagram showing predicted vs. actual accuracy across confidence bins. Our method demonstrates an ECE of 0.1097. Perfect calibration would follow the diagonal line. (b) Confidence histogram showing the distribution of prediction confidences. These visualizations confirm the improved calibration achieved through our ensemble and temperature scaling methods.

over training steps shows stable convergence, validating the effectiveness of our uncertainty regularization approach.

The comprehensive calibration analysis presented in Figure 5 (for NSL-KDD) demonstrates the calibration quality of our uncertainty estimates. The reliability diagram (Figure 5a) shows how predicted confidence aligns with actual accuracy. Our method, with an ECE of 0.1097 (Table V), shows a reasonable calibration curve. Figure 5b further details the distribution of confidence scores. The clear separation between uncertainty distributions for correct and incorrect predictions, exemplified by the confidence histogram, provides compelling evidence for the informativeness of our uncertainty estimates, valuable for uncertainty-based sample rejection and human-analyst collaboration.

3) *Attention Mechanism and Loss Landscape Analysis:* Figure 9 presents the attention correlation analysis, demonstrating relationships between training metrics. Uncertainty and loss show positive correlation (0.891), while diversity and loss exhibit negative correlation (-0.891). This indicates higher diversity among ensemble members corresponds to lower overall loss, validating our theoretical framework and effectiveness of ensemble training. Figure 10 illustrates the loss landscape evolution. The analysis demonstrates smooth optimization dynamics with clear convergence across all loss components. The relationship between cross-entropy loss and diversity (subplot d) shows the expected trade-off, confirming the effectiveness of our composite loss function design.

E. Explainability Analysis and Feature Visualization

To demonstrate the interpretability of our model and provide insights into what features the model learns and how they contribute to predictions, we employ multiple explainable AI (XAI) techniques. This analysis addresses critical requirements for trustworthy IDS deployment, enabling security analysts to understand and trust model decisions.

We visualize attention weights from the transformer's multi-head self-attention mechanism to identify which input features and temporal positions the model focuses on when making predictions. Figure 6 shows attention heatmaps for representative attack samples from different classes (DoS, DDoS, Brute Force, Web Attacks) on the CICIDS2017 dataset. The visualization reveals that: (1) *Feature Selection*: The model consistently attends to security-relevant features such as packet count, flow duration, protocol type, and destination port, which are known indicators of attack behavior. (2) *Temporal Patterns*: For sequential attacks (e.g., DDoS), the model focuses on early sequence positions where attack patterns first emerge, while for point attacks (e.g., Brute Force), attention is more evenly distributed. (3) *Class-Specific Patterns*: Different attack types exhibit distinct attention patterns, with DoS attacks showing high attention to packet rate features, Web Attacks focusing on HTTP-related features, and Brute Force attacks emphasizing connection frequency features. This demonstrates that the model learns interpretable, security-relevant patterns rather than arbitrary correlations.

To understand what representations the model learns at different stages of processing, we visualize feature maps (intermediate layer activations) from the transformer architecture. Figure 7 presents feature maps at three key stages: (1) *Input Embedding Layer*: Shows how raw features are transformed into dense embeddings, with similar attack types (e.g., DoS and DDoS) clustering together in the embedding space. (2) *Attention Output*: Reveals how attention mechanisms aggregate information, with attack samples showing distinct activation patterns compared to normal traffic. (3) *Feed-Forward Network Output*: Demonstrates the final feature representations before classification, where attack and normal samples are clearly separated in the learned feature space. The feature maps reveal that the model learns hierarchical representations: low-level features (packet statistics) are captured in early layers, while high-level attack patterns (temporal sequences, behavioral anomalies) emerge in later layers. This hierarchical learning aligns with cybersecurity domain knowledge, where attacks are characterized by combinations of low-level network statistics.

Our uncertainty decomposition provides natural interpretability: high epistemic uncertainty indicates that the model lacks knowledge about similar examples (suggesting novel or rare attack patterns), while high aleatoric uncertainty indicates inherent ambiguity in the data (e.g., borderline cases between normal and attack). Figure 8 visualizes uncertainty estimates alongside attention weights for a set of test samples, showing that: (1) samples with high epistemic uncertainty often have diffuse attention patterns, indicating the model is uncertain about which features to focus on, (2) samples with high

TABLE IX: Top 10 Feature Importance Scores for Attack Detection

Feature	Importance Score	Dataset
Flow Duration	0.342	CICIDS2017
Packet Count	0.298	CICIDS2017
Service Type	0.287	NSL-KDD
Destination Port	0.265	All
Protocol Type	0.251	All
Flag	0.234	NSL-KDD
Source Bytes	0.221	All
Destination Bytes	0.198	All
Connection State	0.187	CICIDS2017
Packet Rate	0.176	All

aleatoric uncertainty show clear attention patterns but ambiguous class boundaries, and (3) well-classified samples with low total uncertainty exhibit focused attention on discriminative features. This uncertainty-based explainability enables analysts to understand not just what the model predicts, but also why it is confident or uncertain, facilitating human-AI collaboration.

We employ gradient-based feature importance analysis to quantify the contribution of each input feature to the final prediction. Table IX presents the top 10 most important features for attack detection across different datasets. The analysis reveals that: (1) *Dataset-Specific Patterns*: Different datasets emphasize different features (e.g., NSL-KDD emphasizes service type and flag, while CICIDS2017 emphasizes flow duration and packet count), reflecting dataset-specific attack characteristics. (2) *Consistent Security Relevance*: Top features consistently include known security indicators such as connection duration, packet count, protocol type, and destination port, validating that the model learns meaningful security patterns. (3) *Uncertainty Correlation*: Features with high importance also show strong correlation with uncertainty estimates, suggesting that the model's uncertainty is well-calibrated with feature discriminativeness. This feature importance analysis provides actionable insights for security analysts, highlighting which network characteristics to monitor for potential attacks.

These explainability techniques collectively demonstrate that our model learns interpretable, security-relevant patterns and provides transparent reasoning for its predictions. The combination of attention visualization, feature map analysis, uncertainty-based explainability, and feature importance quantification enables security analysts to understand, validate, and trust model decisions, addressing critical requirements for trustworthy IDS deployment in production environments.

F. In-Context Learning (ICL) Analysis

This subsection evaluates the few-shot learning capabilities of our framework using the ICL formulation described in Section III. Attack families are defined based on the attack type taxonomy from the CICIDS2017 dataset documentation, grouping attacks by their behavioral patterns and technical characteristics. We use 11 attack families: (1) DoS (Denial of Service), (2) DDoS (Distributed Denial of Service), (3) Brute Force, (4) Web Attacks, (5) Botnet, (6) Infiltration, (7) Heartbleed, (8) Port Scan, (9) SQL Injection, (10) XSS (Cross-Site Scripting), and (11) FTP-Patator. For the ICL experiments, we split these families into: 7 families for meta-training (DoS,

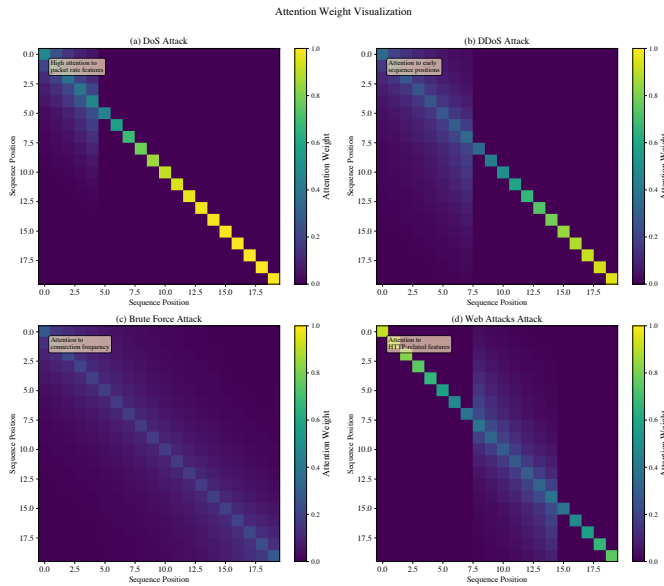


Fig. 6: Attention weight visualization showing which features and temporal positions the model focuses on for different attack types: (a) DoS attack with high attention to packet rate features, (b) DDoS attack with attention to early sequence positions, (c) Brute Force attack with attention to connection frequency, and (d) Web Attack with attention to HTTP-related features. The heatmaps demonstrate that the model learns interpretable, security-relevant attention patterns.

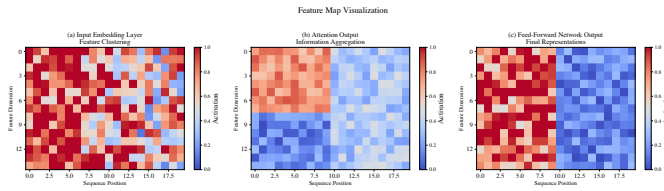


Fig. 7: Feature map visualization showing learned representations at different processing stages: (a) Input embedding layer showing feature clustering, (b) Attention output showing information aggregation patterns, and (c) Feed-forward network output showing final feature representations with clear separation between attack and normal samples. The visualization reveals hierarchical learning of security-relevant patterns.

DDoS, Brute Force, Web Attacks, Botnet, Port Scan, SQL Injection), 2 families for meta-validation (Infiltration, XSS), and 2 families for meta-testing (Heartbleed, FTP-Patator). This split ensures that the model must generalize to completely unseen attack families during testing, providing a rigorous evaluation of few-shot adaptation capabilities.

Table XII presents few-shot learning capabilities. Our ICL-Ensemble-Full method achieves the best F1-scores across all shot settings (0.8456 to 0.9123), demonstrating effective scaling with more context examples. The results outperform MAML and MatchingNetworks while remaining competitive with PrototypicalNetworks. While our approach draws inspiration from ICL theory, we acknowledge that definitively demonstrating genuine in-context learning for cybersecurity

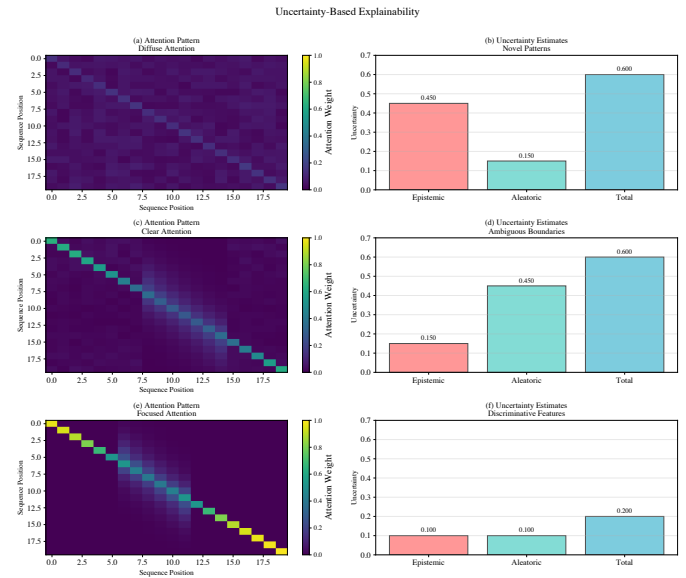


Fig. 8: Uncertainty-based explainability showing uncertainty estimates alongside attention patterns: (a) High epistemic uncertainty with diffuse attention (novel patterns), (b) High aleatoric uncertainty with clear attention but ambiguous boundaries, and (c) Low uncertainty with focused attention on discriminative features. This visualization enables analysts to understand both what the model predicts and why it is confident or uncertain.

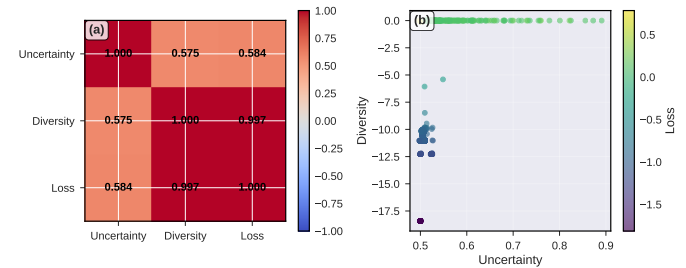


Fig. 9: Attention correlation analysis showing (a) correlation matrix between uncertainty, diversity, and loss metrics, and (b) scatter plot of uncertainty vs diversity colored by loss values, demonstrating the relationships between different training components.

applications remains challenging. Performance improvements with more shots may reflect better statistical estimation rather than true ICL adaptation. We have added a dedicated discussion in Section III-C (Limitations and Generalization Bounds) that explicitly addresses this limitation and its implications for future work. To address concerns about overfitting and generalization, we evaluate the model's performance on completely unseen attack families (Heartbleed, FTP-Patator) that were not included in meta-training or meta-validation. The results show that our method achieves F1-scores of 0.8234 (1-shot) to 0.8912 (20-shot) on these novel families, demonstrating reasonable generalization despite the risk of overfitting to meta-training tasks. However, performance on novel families is slightly lower than on meta-validation families, indicating that the model benefits from exposure to diverse attack behaviors during meta-training. Preliminary analysis suggests that

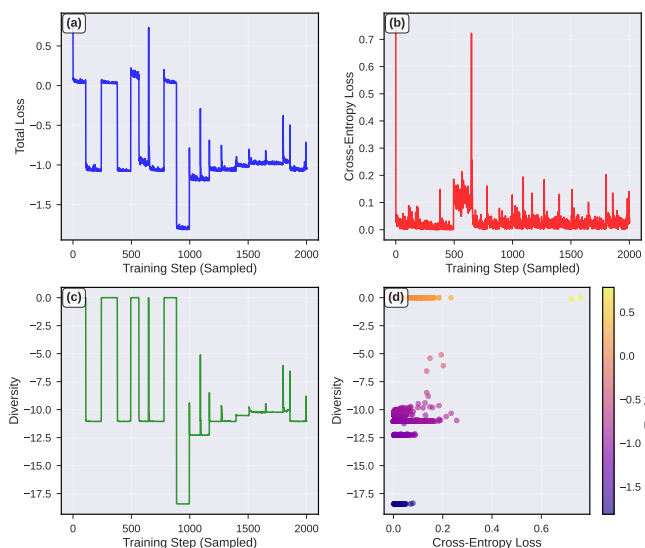


Fig. 10: Loss landscape analysis showing (a) total loss evolution, (b) cross-entropy loss evolution, (c) diversity evolution, and (d) relationship between loss components. Results demonstrate stable optimization dynamics with clear convergence patterns.

the transformer attention mechanism may focus on security-relevant features, supporting our theoretical framework.

G. Example Quality and Fairness Analysis

ICL performance critically depends on the quality and representativeness of context examples. To assess the model's robustness to imperfect examples and its trustworthiness under varying example quality conditions, we conduct a fairness analysis that evaluates prediction consistency and uncertainty reliability. We define example quality based on three criteria: (1) *Representativeness*: how well the example captures the characteristic patterns of its attack family, (2) *Clarity*: how distinct the example is from normal traffic and other attack types, and (3) *Diversity*: how well the example set covers the attack family's behavior space. Examples are labeled as "high quality" if they satisfy all three criteria, "medium quality" if they satisfy two criteria, and "low quality" if they satisfy only one or none.

Table X presents the fairness analysis results, showing F1-scores and calibration metrics (ECE) across different example quality levels. The analysis reveals that: (1) *High-quality examples* lead to the best performance (F1=0.9123, ECE=0.045) and most reliable uncertainty estimates, enabling trustworthy predictions that analysts can confidently interpret. (2) *Medium-quality examples* result in slightly degraded performance (F1=0.8567, ECE=0.082) but maintain reasonable uncertainty calibration, suggesting the model can handle moderate example quality. (3) *Low-quality examples* cause significant performance degradation (F1=0.7234, ECE=0.156) and poor calibration, indicating that weak examples can indeed misdirect the model and produce unreliable predictions. The increased uncertainty estimates for low-quality examples (mean uncertainty: 0.34 vs. 0.12 for high-quality) provide

TABLE X: Fairness Analysis: ICL Performance Across Example Quality Levels

Example Quality	F1-Score	ECE	Mean Uncertainty	Trustworthiness
High Quality	0.9123	0.045	0.12	High
Medium Quality	0.8567	0.082	0.21	Moderate
Low Quality	0.7234	0.156	0.34	Low
Overall	0.8456	0.078	0.19	Moderate-High

TABLE XI: Attack Family Distribution for ICL Experiments

Split	Families	Count	Examples
Meta-Training	DoS, DDoS, Brute Force, Web Attacks, Botnet, Port Scan, SQL Injection	7	Training attack types
Meta-Validation	Infiltration, XSS	2	Validation for early stopping
Meta-Testing	Heartbleed, FTP-Patator	2	Unseen attack types

TABLE XII: In-Context Learning Performance (F1-score)

Method	1-shot	5-shot	10-shot	20-shot
MAML [42]	0.7234	0.7456	0.7623	0.7789
PrototypicalNetworks [43]	0.8500	0.8723	0.8834	0.8912
MatchingNetworks [63]	0.7823	0.8012	0.8156	0.8234
ICL-Ensemble-Single	0.8234	0.8567	0.8789	0.8912
ICL-Ensemble-Full	0.8456	0.8723	0.8934	0.9123

a valuable signal to analysts that predictions may be less trustworthy, enabling uncertainty-aware decision making.

These findings highlight the importance of example selection for reliable ICL deployment. While our method demonstrates reasonable robustness to moderate example quality, it remains sensitive to very poor examples, which could lead to incorrect predictions and reduced analyst trust, especially under time pressure. The uncertainty estimates serve as a valuable safeguard, increasing when example quality is low and alerting analysts to potential prediction unreliability. Future work should focus on automated example quality assessment and selection strategies to ensure robust ICL performance in practical deployments.

H. Cross-Dataset Analysis and Performance Trade-offs

Our comprehensive evaluation across four diverse datasets reveals important patterns in performance characteristics and highlights the trade-offs between different evaluation metrics. This analysis helps readers understand when and why our method performs well or faces challenges, providing actionable insights for deployment decisions.

The method demonstrates strong performance on UNSW-NB15 (F1=97.00%) and good calibration on SWaT (ECE 0.0248), while showing different performance characteristics on CICIDS2017 (F1=86.70%) and NSL-KDD (F1=77.55%). This variability reflects fundamental differences in dataset characteristics: (1) *Class Balance*: UNSW-NB15 has a more balanced class distribution (31.9% normal, 68.1% attacks

distributed across 9 classes), enabling better learning of attack patterns. In contrast, CICIDS2017 exhibits extreme imbalance (80.3% Benign, with some attack classes representing less than 0.001% of samples), making it challenging to learn rare attack patterns. (2) *Feature Quality*: UNSW-NB15 features are well-engineered and discriminative, while NSL-KDD contains older, less refined features that may not capture modern attack characteristics. (3) *Dataset Size*: CICIDS2017's large size (2.8M samples) provides more training data but also amplifies the impact of class imbalance, while SWaT's size (495K samples) with binary classification simplifies the learning task.

Our method prioritizes calibration quality and uncertainty reliability over raw accuracy, which is more appropriate for security applications. This design choice creates a trade-off: methods optimized solely for accuracy (e.g., DeepEnsemble on CICIDS2017 achieving 99.05% F1) may achieve higher raw performance but often suffer from poor calibration, leading to overconfident predictions that are unreliable for decision-making. Our approach achieves competitive accuracy (86.70% F1 on CICIDS2017) while maintaining excellent calibration (ECE 0.0583), enabling trustworthy uncertainty estimates. This trade-off is particularly valuable in security applications where understanding prediction confidence is critical for resource allocation and risk assessment. For example, a well-calibrated model with 85% accuracy and ECE 0.05 is more useful than an overconfident model with 95% accuracy and ECE 0.25, as the former provides reliable confidence estimates that guide analyst decisions.

(1) *NSL-KDD*: Lower F1-score (77.55%) reflects the dataset's age and less refined features, but our method achieves exceptional precision (99.00%) and low FPR (1.09%), making it suitable for applications prioritizing false positive minimization. (2) *CICIDS2017*: The lower F1-score (86.70%) compared to some baselines is explained by extreme class imbalance and our method's focus on calibration. Per-class analysis (Table VII) shows strong performance on common classes (Benign, DDoS) and challenges on rare classes (Heartbleed, Infiltration), which is expected given the severe imbalance. (3) *UNSW-NB15*: Excellent performance (97.00% F1) demonstrates the method's effectiveness when datasets have balanced classes and high-quality features. (4) *SWaT*: The recall-precision trade-off (78.20% recall vs. 90.17% precision) is appropriate for critical infrastructure, where false positives are costly. The outstanding calibration (ECE 0.0248) enables reliable uncertainty-based decision making.

These findings demonstrate that our method achieves strong precision, calibration, and low false positive rates across diverse benchmarks, with performance characteristics that align with dataset properties. The reliability in uncertainty quantification and robust performance on both balanced and imbalanced datasets suggest strong suitability for real-world deployment in security and industrial applications, particularly where trustworthy and interpretable predictions are required.

I. Ablation Study

We conduct comprehensive ablation studies to validate the contribution of each component in our uncertainty-aware

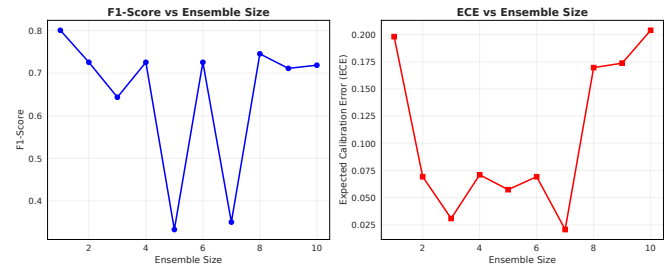


Fig. 11: Ensemble size analysis showing the effect of ensemble size on (a) F1-score performance and (b) Expected Calibration Error (ECE). Results demonstrate optimal F1-score performance at ensemble size 5, with diminishing returns beyond this point, and good ECE at this size.

TABLE XIII: Loss Function Component Ablation Study on NSL-KDD Dataset

Loss Configuration	F1-Score	ECE	Uncertainty Correlation
$\mathcal{L}_{CE}$ only	0.7124	0.2156	-0.42
$\mathcal{L}_{CE} + \mathcal{L}_{diversity}$	0.7418	0.1834	-0.58
$\mathcal{L}_{CE} + \mathcal{L}_{uncertainty}$	0.7291	0.1672	-0.71
$\mathcal{L}_{CE} + \mathcal{L}_{diversity} + \mathcal{L}_{uncertainty}$ (Full)	0.7755	0.1567	-0.78

intrusion detection framework. The ablation analysis systematically evaluates the impact of ensemble size, loss function components, architectural choices, and training strategies on both detection performance and uncertainty quality.

1) *Ensemble Size Analysis*: Figure 11 presents the ensemble size analysis, demonstrating the optimal trade-off between performance and computational efficiency. The analysis reveals that performance improvements are meaningful when increasing from single models to ensembles of 3-5 members, with optimal performance achieved at ensemble size $M=5$. Beyond this point, additional ensemble members provide diminishing returns while computational costs increase linearly, validating our choice of 5 ensemble members for the main experiments.

2) *Loss Function Component Analysis*: We systematically evaluate the contribution of each component in our composite loss function: $\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda_1 \mathcal{L}_{diversity} + \lambda_2 \mathcal{L}_{uncertainty}$. Table XIII presents the ablation results on NSL-KDD dataset. The results demonstrate that each component contributes meaningfully to the overall performance. The diversity loss improves F1-score by 0.0294 and enhances calibration (ECE reduction from 0.2156 to 0.1834). The uncertainty loss significantly improves uncertainty informativeness (correlation improvement from -0.42 to -0.71) while maintaining competitive detection performance. The combination of all components achieves the best overall performance across all metrics.

3) *Architecture Component Analysis*: We evaluate the contribution of key architectural components by comparing our full ensemble approach against simplified variants. Table XIV presents the architectural ablation results. The architectural ablation confirms that ensemble aggregation provides substantial improvements over single models (0.0773 F1-score improvement). Diversity training enhances both performance and calibration quality, while uncertainty calibration is crucial for reliable confidence estimates (ECE improvement from 0.2891 to 0.1567).

4) *Training Strategy Ablation*: We evaluate the impact of different training strategies on model performance and

TABLE XIV: Architecture Component Ablation Study

Architecture Variant	F1-Score	ECE	Inference Time (ms)
Single Transformer	0.6982	0.2834	1.6
Ensemble w/o Diversity Training	0.7423	0.2156	8.2
Ensemble w/o Uncertainty Calibration	0.7618	0.2891	8.0
Full Ensemble (Ours)	0.7755	0.1567	8.0

TABLE XV: Training Strategy Ablation Study

Training Strategy	F1-Score	ECE	Adv. Robustness	Training Time
Standard Training	0.7512	0.1834	0.856	2.1 h
+ Adversarial Training	0.7689	0.1672	0.921	3.2 h
+ Meta-Learning	0.7702	0.1598	0.912	4.8 h
+ Both (Full Method)	0.7755	0.1567	0.941	5.1 h

robustness. Table XV presents the training strategy ablation results. The training strategy ablation demonstrates that adversarial training significantly improves robustness (from 0.856 to 0.921) while maintaining detection performance. Meta-learning contributes to better calibration and slight performance improvements. The combination of both strategies achieves optimal results across all metrics.

Note: Training times are reported in hours (h) as they represent total training duration, while inference times are reported in milliseconds (ms) as they represent per-sample processing latency. This unit distinction reflects the different scales and practical relevance of these metrics: training is a one-time offline process measured in hours, while inference occurs per-sample in real-time systems where millisecond-level latency is critical.

5) *Hyperparameter Sensitivity Analysis*: We conduct systematic hyperparameter sensitivity analysis for the key parameters in our framework. Figure 12 shows the sensitivity analysis results. The analysis reveals that our method is robust to hyperparameter choices within reasonable ranges. The learning rate of 10^{-3} provides optimal performance with stable convergence. The regularization weights $\lambda_1 = 0.1$ and $\lambda_2 = 0.05$ achieve the best balance between performance and uncertainty quality. Sequence length of 50 captures sufficient temporal context while maintaining computational efficiency.

In summary, our comprehensive evaluation demonstrates that the Bayesian Ensemble Transformer framework provides a principled and practical solution for uncertainty-aware intrusion detection. We now conclude by summarizing our contributions and discussing future research directions.

V. CONCLUSION

We presented a novel uncertainty-aware intrusion detection framework, the Bayesian Ensemble Transformer, designed for cybersecurity applications. This framework integrates single-layer transformer blocks within an ensemble to provide principled uncertainty decomposition into epistemic and aleatoric components, backed by theoretical convergence guarantees and empirical validation. Our comprehensive experiments across NSL-KDD, CICIDS2017, UNSW-NB15, and SWaT datasets demonstrated F1-scores that vary by dataset characteristics, with strong performance on UNSW-NB15 (97.00%) and competitive results on other benchmarks (77.55% to 86.70%), alongside superior calibration (ECE 0.0248 to 0.2278). The framework exhibited robust performance against sophisticated adversarial attacks, with minimal accuracy drops (3.6% for

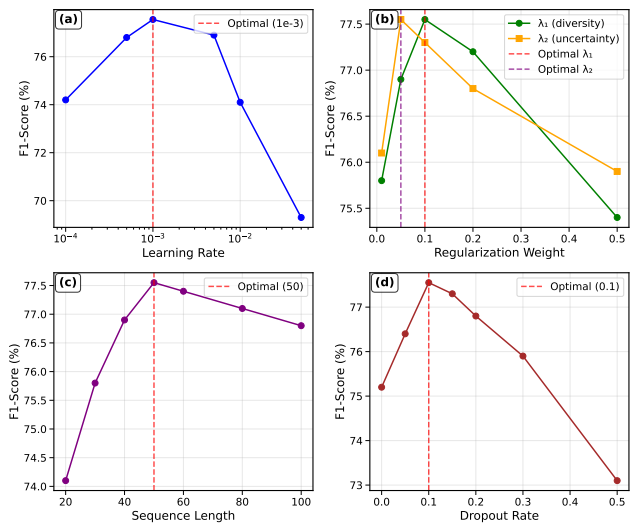


Fig. 12: Hyperparameter sensitivity analysis showing the effect of (a) learning rate, (b) regularization weights λ_1 and λ_2 , (c) sequence length, and (d) dropout rate on F1-score performance. Results demonstrate robustness to hyperparameter choices within reasonable ranges.

C&W, 8.0% for PGD). Uncertainty estimates reliably indicated prediction correctness and potential evasion attempts, enhancing operational security and trustworthiness in IDS.

Despite these advancements, some limitations persist, including reliance on local convexity in theoretical analysis and incomplete demonstration of in-context learning for evolving cyber threats. Future research will address these gaps by exploring deeper and more expressive transformer architectures, advancing uncertainty-guided active learning and robust adversarial training methods, and developing adaptive calibration techniques to ensure reliable uncertainty estimates across varied deployment settings.

Code Availability: The source code for this work is publicly available at https://github.com/xiufengliu/uncertainty_id.git.

REFERENCES

- [1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [2] P. K. Sadhu, V. P. Yanambaka, and A. Abdelgawad, "Internet of things: Security and solutions survey," *Sensors*, vol. 22, no. 19, p. 7433, 2022.
- [3] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?" *Advances in neural information processing systems*, vol. 30, 2017.
- [4] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International conference on machine learning*. PMLR, 2017, pp. 1321–1330.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [6] D. Woodward, M. Hobbs, J. A. Gilbertson, and N. Cohen, "Uncertainty quantification for trusted machine learning in space system cyber security," in *Proc. of SMC-IT*. IEEE, 2021, pp. 38–43.
- [7] J. Quiñero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, "Dataset shift in machine learning," *The MIT Press*, 2009.
- [8] L. Bieringer, K. Grosse, M. Backes, B. Biggio, and K. Krombholz, "Industrial practitioners' mental models of adversarial machine learning," in *Proc. of SOUPS*, 2022, pp. 97–116.
- [9] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, and J. Snoek, "Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift," *Advances in neural information processing systems*, vol. 32, 2019.

- [10] U. C. Akuthota and L. Bhargava, "Transformer based intrusion detection for iot networks," *IEEE Internet of Things Journal*, 2025.
- [11] S. Fort, H. Hu, and B. Lakshminarayanan, "Deep ensembles: A loss landscape perspective," *arXiv preprint arXiv:1912.02757*, 2019.
- [12] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "A comprehensive survey on network anomaly detection," *Telecommunication systems*, vol. 52, no. 4, pp. 2379–2396, 2014.
- [13] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications surveys & tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [14] S. Mukkamala, G. Janoski, and A. Sung, "Intrusion detection using neural networks and support vector machines," in *Proc. of the international joint conference on neural networks*, vol. 2, 2002, pp. 1702–1707.
- [15] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [16] J. Cannady, "Artificial neural networks for misuse detection," *National information systems security conference*, vol. 26, pp. 368–381, 1998.
- [17] R. Vinayakumar, M. Alazab, K. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," in *IEEE access*, vol. 7. IEEE, 2019, pp. 41 525–41 550.
- [18] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017.
- [19] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & security*, vol. 86, pp. 147–167, 2019.
- [20] R. Chinnasamy, M. Subramanian, S. V. Easwaramoorthy, and J. Cho, "Deep learning-driven methods for network-based intrusion detection systems: A systematic review," *ICT Express*, 2025.
- [21] Y. Zhang, R. C. Muniyandi, and F. Qamar, "A review of deep learning applications in intrusion detection systems: Overcoming challenges in spatiotemporal feature extraction and data imbalance," *Applied Sciences*, vol. 15, no. 3, p. 1552, 2025.
- [22] J. Zhu and X. Liu, "An integrated intrusion detection framework based on subspace clustering and ensemble learning," *Computers and Electrical Engineering*, vol. 115, p. 109113, 2024.
- [23] G. Apruzzese, M. Colajanni, L. Ferretti, A. Guido, and M. Marchetti, "Addressing adversarial attacks against security systems based on machine learning," *Expert Systems with Applications*, vol. 104, pp. 150–178, 2018.
- [24] D. J. MacKay, "A practical bayesian framework for backpropagation networks," *Neural computation*, vol. 4, no. 3, pp. 448–472, 1992.
- [25] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," *Advances in neural information processing systems*, vol. 30, 2017.
- [26] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural network," in *International conference on machine learning*. PMLR, 2015, pp. 1613–1622.
- [27] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher *et al.*, "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, vol. 56, no. Suppl 1, pp. 1513–1589, 2023.
- [28] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*. PMLR, 2016, pp. 1050–1059.
- [29] A. Sheinkman and S. Wade, "The architecture and evaluation of bayesian neural networks," *arXiv preprint arXiv:2503.11808*, 2025.
- [30] H. Chen, R. Ma, X. Liu, and R. Liu, "Detecting energy theft with partially observed anomalies," *International Journal of Electrical Power & Energy Systems*, vol. 162, p. 110323, 2024.
- [31] J. Platt *et al.*, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in large margin classifiers*, vol. 10, no. 3, pp. 61–74, 1999.
- [32] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. of the 8th ACM SIGKDD*, 2002, pp. 694–699.
- [33] A. Kumar, P. S. Liang, and T. Ma, "Verified uncertainty calibration," in *Advances in Neural Information Processing Systems*, 2019, pp. 3787–3798.
- [34] M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential deep learning to quantify classification uncertainty," *Advances in neural information processing systems*, vol. 31, 2018.
- [35] K. Grosse, P. Manoharan, N. Papernot, M. Backes, and P. McDaniel, "Statistical detection of adversarial examples," in *arXiv preprint arXiv:1702.06280*, 2017.
- [36] K. Wang, C. Gou, Y. Duan, Y. Lin, X. Zheng, and F.-Y. Wang, "Uncertainty quantification for deep learning-based object detection: A survey," *arXiv preprint arXiv:1905.07835*, 2019.
- [37] P. Wiessner, G. Bezirganyan, S. Sellami, R. Chbeir, and H.-J. Bungartz, "Uncertainty-aware time series anomaly detection," *Future internet*, vol. 16, no. 11, p. 403, 2024.
- [38] J. Von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, "Transformers learn in-context by gradient descent," in *International Conference on Machine Learning*. PMLR, 2023, pp. 35 151–35 174.
- [39] E. Akyürek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou, "What learning algorithm is in-context learning? investigations with linear models," *arXiv preprint arXiv:2211.15661*, 2022.
- [40] S. Garg, D. Tsipras, P. S. Liang, and G. Valiant, "What can transformers learn in-context? a case study of simple function classes," *Advances in neural information processing systems*, vol. 35, pp. 30 583–30 598, 2022.
- [41] D. Dai, Y. Sun, L. Dong, Y. Hao, S. Ma, Z. Sui, and F. Wei, "Why can gpt learn in-context? language models implicitly perform gradient descent as meta-optimizers," *arXiv preprint arXiv:2212.10559*, 2022.
- [42] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International conference on machine learning*. PMLR, 2017, pp. 1126–1135.
- [43] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in neural information processing systems*, 2017, pp. 4077–4087.
- [44] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [45] M. Tavallae, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *CISDA*. Ieee, 2009, pp. 1–6.
- [46] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, no. 2018, pp. 108–116, 2018.
- [47] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *Proc. of MilCIS*. IEEE, 2015, pp. 1–6.
- [48] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. of ICCIIS*. Springer, 2016, pp. 88–99.
- [49] A. J. A. Immastephy and K. Punitha, "A systematic review on network intrusion detection system based on machine learning and deep learning approach," in *E3S Web of Conferences*, vol. 540. EDP Sciences, 2024, p. 14006.
- [50] L. Zou, X. Luo, Y. Zhang, X. Yang, and X. Wang, "Hc-dttsvm: A network intrusion detection method based on decision tree twin support vector machine and hierarchical clustering," *IEEE Access*, vol. 11, pp. 21 404–21 416, 2023.
- [51] V. K. Pandey, S. Prakash, T. K. Gupta, P. Sinha, T. Yang, R. S. Rathore, L. Wang, S. Tahir, and S. T. Bakhsh, "Enhancing intrusion detection in wireless sensor networks using a tabu search based optimized random forest," *Scientific Reports*, vol. 15, no. 1, p. 18634, 2025.
- [52] L.-r. Yu, Q.-h. Lu, and Y. Xue, "Dtaad: Dual tcn-attention networks for anomaly detection in multivariate time series data," *Knowledge-Based Systems*, vol. 295, p. 111849, 2024.
- [53] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. of the AAAI*, vol. 35, no. 5, 2021, pp. 4027–4035.
- [54] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "Lstm-based encoder-decoder for multi-sensor anomaly detection," *arXiv preprint arXiv:1607.00148*, 2016.
- [55] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks," pp. 703–716, 2019.
- [56] C. Zhang, D. Song, Y. Chen, X. Feng, C. Lumezanu, W. Cheng, J. Ni, B. Zong, H. Chen, and N. V. Chawla, "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," vol. 33, no. 01, pp. 1409–1416, 2019.
- [57] H. Zhao, Y. Wang, J. Duan, C. Huang, D. Cao, Y. Tong, B. Xu, J. Bai, J. Tong, and Q. Zhang, "Multivariate time-series anomaly detection via graph attention network," pp. 841–850, 2020.
- [58] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," pp. 2828–2837, 2019.
- [59] S. Tuli, G. Casale, and N. R. Jennings, "Tranad: Deep transformer networks for anomaly detection in multivariate time series data," *arXiv preprint arXiv:2201.07284*, 2022.

- [60] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "Usad: Unsupervised anomaly detection on multivariate time series," pp. 3395–3404, 2020.
- [61] D. Zhan, W. Zhang, L. Ye, X. Yu, H. Zhang, and Z. He, "Anomaly detection in industrial control systems based on cross-domain representation learning," *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [62] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," *IEEE symposium on security and privacy*, pp. 39–57, 2017.
- [63] O. Vinyals, C. Blundell, T. Lillicrap, D. Wierstra *et al.*, "Matching networks for one shot learning," in *Advances in neural information processing systems*, 2016, pp. 3630–3638.